SWE5006 Practice Project · Graduate Certificate in Designing Modern Software Systems

Speaker Script and Live Demo Run Book

BlocBuy — Team 8 · Monday 26 October 2026, 16:30–17:10 SGT (Zoom)

Responsible	Yu Qianshuo (Yorik), presentation lead and storyline (#464); Lim Jiechao, slide master.
Specification	`docs/swe5006/presentation/RUBRIC.md`: every slide records the rubric rows it covers; the coverage tables in the run sheet are derived from them.
Companion files	`swe5006-presentation.html` — the deck (37 slides + backups BX, B1–B16 and the 1:1 diagram detail pages `D9-1` … `D29-3`; Chrome or Edge; **S** notes, **T** timer, **O** overview, **F** fullscreen, **B** backup index) · `swe5006-presentation.pdf` — the same deck for submission.
Facts as of	21 Oct 2026. Dev health `UP`; #223 and #517 open, so no payment has been confirmed and no refund proven on dev yet.

Markers

`[FILL]` value that does not exist yet; never present a guess · `[VERIFY]` claim to check against the repository, report or run logs · `[CONFIRM]` needs team agreement. They must all be gone after the 25 Oct rehearsal.

Run sheet and coverage check

Slide-by-slide run sheet

Clock	Slot	Slide	Title	Speaker	Item	Est.
16:30	5 s	1	BlocBuy	Lim	1a	4 s
16:30:05	25 s	2	Group buying fails when buyers cannot commit together	Lim	1a	24 s
16:30:30	30 s	3	Scope: nine use cases, six quality attributes with targets	Lim	1b	30 s
16:31	30 s	4	Promised, delivered, changed, dropped: status on 21 October	Lim	1c	29 s
16:31:30	15 s	5	Milestones achieved and effort per member	Lim	1d	13 s
16:31:45	15 s	6	Ownership: one use case and one design problem each	Lim	1d	13 s
16:32	45 s	7	Four sprints: the Definition of Done reached, and what each retro changed	Lim	1e	42 s
16:32:45	15 s	8	Agile artefacts: a story card, a burndown, a practice	Lim	1g	14 s
16:33	45 s	9	Logical architecture: COLA layers across six bounded contexts	Eric	2a	25 s
16:33:45	40 s	10	Transition strategies from analysis to design	Eric	2b	20 s
16:34:25	35 s	11	One strategy pair: analysis objects become design classes	Eric	2b	21 s
16:35	30 s	12	Use case diagram: in scope, out of scope and status	Eric	2c	21 s
16:35:30	20 s	13	UC4 join: the analysis model	Eric	2d	19 s
16:35:50	15 s	14	UC4 join: the design class diagram	Eric	2d	14 s
16:36:05	15 s	15	UC4 join: the design sequence, paid join by webhook	Eric	2d	15 s
16:36:20	10 s	16	Team-formation statechart	Eric	2d	9 s
16:36:30	20 s	17	UC3 start a team: Factory Method	Eric	2e	20 s
16:36:50	20 s	18	UC5 to UC6 team outcome: Observer	Yorik	2e	18 s
16:37:10	20 s	19	Notifications: Template Method for outbox transports	Pan	2e	18 s
16:37:30	20 s	20	UC1 pricing: Strategy	Lim	2e	19 s
16:37:50	30 s	21	Design problem: listing rules that depend on its state	Liu Jin	2e	27 s
16:38:20	25 s	22	After: the State pattern, and why the team uses a table	Liu Jin	2e	23 s
16:38:45	15 s	23	Database schema: core entities and mapping decisions	Liu Jin	2f	14 s
16:39	110 s	24	Testing on the Agile Testing Quadrants	Liu Jin	3a	102 s
16:40:50	70 s	25	Scripted performance test: tool, scenario, target	Liu Jin	3a	65 s
16:42	60 s	26	Source code and artefacts: one trunk, everything traceable to a commit	Pan	4a	58 s
16:43	60 s	27	Continuous integration: pull-request jobs and post-merge jobs	Pan	4b	56 s
16:44	45 s	28	Security controls by pipeline stage, and what each one catches	Pan	4d	42 s
16:44:45	60 s	29	Delivery: one environment, the code owner approves, the deploy verifies itself	Pan	4b	60 s

16:45:45	45 s	30	Technical findings: three issues, their root causes and fixes	Pan	4c	43 s
16:46:30	330 s	31	Live: one change from pull request to dev	Pan	4e	21 s
16:52	30 s	32	System demo: two buyers, one bloc, and the qualities it shows	Yorik	5a	26 s
16:52:30	510 s	33	Live: two buyers, one bloc	Yorik	5c	13 s
17:01	120 s	34	Performance against the target: the scripted k6 run	Yorik	5b	73 s
17:03	30 s	35	Scalability: a stated trade-off, not a claim	Yorik	5b	27 s
17:03:30	30 s	36	Maintainability: the pipeline you just saw, and the seams in the code	Yorik	5b	30 s
17:04	Q&A	37	Thank you	Yorik	5b	

Segment talk budget vs slots vs estimated speaking time: §1 180 s / 180 s / 169 s · §2 360 s / 360 s / 283 s · §3 180 s / 180 s / 167 s · §4 600 s / 600 s / 280 s · §5 720 s / 720 s / 169 s.

34 minutes scripted, 16:30–17:04; the 6 minutes to 17:10 are the buffer for questions asked during the talk (rubric G1–G3). §1 runs 3 minutes and §2 6 minutes so that each member presents their own design problem (rubric G4, time budget).

Interruption policy

- A lecturer may ask at any moment. The current speaker answers in **one or two sentences** (decision → reason → evidence or limitation), says “back to …” and continues from the same slide.
- If the owner is someone else, the speaker says so and Lim posts `Q: <question> → owner` in the backchannel; parked questions are answered first in the buffer.
- Hard checkpoints: 16:33 Eric, 16:39 Liu Jin, 16:42 Pan, 16:52 Yorik.

Handovers (rubric G4: rehearse each one)

From slide	Speakers
9	Lim Jiechao → Wang Ziang (Eric)
18	Wang Ziang (Eric) → Yu Qianshuo (Yorik)
19	Yu Qianshuo (Yorik) → Panpakorn Siripanich (Pan)
20	Panpakorn Siripanich (Pan) → Lim Jiechao
21	Lim Jiechao → Liu Jin
26	Liu Jin → Panpakorn Siripanich (Pan)
32	Panpakorn Siripanich (Pan) → Yu Qianshuo (Yorik)

Every guideline item has slides and a speaker

Item	Guideline requirement (verbatim)	Slides	Speaker
1a	Explain the context and the business problem that the solution as a whole is trying to achieve.	1, 2	Lim Jiechao
1b	Explain the scope of the project.	3	Lim Jiechao
1c	Explain the current status of the project including outstanding issues.	4	Lim Jiechao

1d	Describe the actual project milestones and the total effort expended by each team member.	5, 6	Lim Jiechao
1e	Explain how many sprints planned, Definition of Done (DoD) achieved.	7	Lim Jiechao
1f	Sprint retrospective & reflection in the next sprint	7	Lim Jiechao
1g	Agile process artifacts like product backlog, user stories, sprint plans, burndown charts, sprint reviews and sprint retrospectives.	8	Lim Jiechao
2a	Software architectural decisions that influence the logical architecture.	9	Wang Ziang
2b	Transition from analysis to design	10, 11	Wang Ziang
2c	Use case diagram	12	Wang Ziang
2d	Analysis and design models	13, 14, 15, 16	Wang Ziang
2e	Design problems and patterns	17, 18, 19, 20, 21, 22	Wang Ziang, Yu Qianshuo, Panpakorn Siripanich, Lim Jiechao, Liu Jin
2f	Database schemas	23	Liu Jin
3a	Types of Testing Conducted	24, 25	Liu Jin
3b	Well structured source code with unit test cases for classes with more complex behaviours.	24	Liu Jin
4a	Describe the strategy to manage the source code and artifacts.	26	Panpakorn Siripanich
4b	Describe the DevSecOps pipeline and procedures.	27, 29	Panpakorn Siripanich
4c	Highlight any technical findings and issues encountered.	30	Panpakorn Siripanich
4d	Design specification of the DevOps pipelines along with the use of appropriate tools with minimal security considerations.	28	Panpakorn Siripanich
4e	Backup: Record a video in advance in case of any technical issues during the live demo.	31	Panpakorn Siripanich
5a	Demonstrate some of the significant use cases of the solution.	32	Yu Qianshuo
5b	Demonstrate how the solution deals with some of the important quality attributes e.g. scalability (such as demonstrating auto-scaling), performance (show using some scripted performance testing), maintainability (show/run the DevOps pipeline).	34, 35, 36, 37	Yu Qianshuo
5c	Backup: Record a video in advance in case of any technical issues during the live demo.	33	Yu Qianshuo

Every rubric row has a slide (rubric checklist)

Rubric row	Slides	Speaker
1.1	1, 2	Lim Jiechao
1.2	3	Lim Jiechao
1.3	4	Lim Jiechao
1.4	5	Lim Jiechao
1.5	6	Lim Jiechao
1.6	7	Lim Jiechao
1.7	7	Lim Jiechao

1.8	8	Lim Jiechao
2.1	9	Wang Ziang
2.2	10, 11, 14, 15	Wang Ziang
2.3	12	Wang Ziang
2.4	11, 13, 14, 15, 16, 22	Wang Ziang, Liu Jin
2.5	17, 18, 19, 20, 21, 22	Wang Ziang, Yu Qianshuo, Panpakorn Siripanich, Lim Jiechao, Liu Jin
2.6	23	Liu Jin
3.1	24	Liu Jin
3.2	25	Liu Jin
3.3	24, 28	Liu Jin, Panpakorn Siripanich
4.1	31	Panpakorn Siripanich
4.2	26	Panpakorn Siripanich
4.3	27, 28	Panpakorn Siripanich
4.4	29	Panpakorn Siripanich
4.5	30	Panpakorn Siripanich
4.6	31	Panpakorn Siripanich
5.1	32, 33	Yu Qianshuo
5.2	32, 33, 35	Yu Qianshuo
5.3	34	Yu Qianshuo
5.4	36	Yu Qianshuo
5.5	33	Yu Qianshuo

Presentation tips checklist

Tip (guidelines page 2)	How this deck meets it
Summarize and highlight the significant achievements. Talk more about the significant and interesting parts of the solution and implementation. Your audience should remember the unique aspects of the solution and not similarities to other solutions.	Depth goes to what is unique: pay-first teams that refund themselves, money safety under concurrency and failure (UC4 design, statechart, design problems), and a pipeline from pull request to dev in minutes.
Manage your time well. Do not present too many slides. Remember that the audience can read the text in the slide so refrain from reading them.	37 main slides in 34 minutes, two of them live demos; slots checked at 130 wpm in the table above; sample wording adds to the slide instead of reading it; details sit in backups B1–B16.
Explain your design in top-down manner. Make sure that your audience understands the requirements, the high-level design and the high-level decisions that you make before drilling into some details.	Problem, scope and quality attributes first (slides 2–3); then architecture (9), transition strategies (10–11), use cases (12), one use case from analysis to design with its statechart (13–16), design problems (17–22), schema (23).
Ensure that the contents on the slides are readable from the distance at which your audience is sitting. This is particular true for face-to-face presentation.	Rubric V1–V7: one idea per slide, diagrams cropped to the part discussed, labels checked on a projector or TV before 26 Oct.

Cue words in the team backchannel (never the Zoom chat)

Message	Meaning	Sent by
`1 MIN`	One minute of talk left in this segment	Lim
`TIME`	Stop; finish the sentence and hand over	Lim
`Q: <text> → name`	A question was parked; the named owner answers it in the buffer (17:04–17:10)	Lim
`BEHIND n / AHEAD n`	The talk is n seconds behind or ahead of the run sheet; the next speaker applies the trims	Lim
`SHARE`	Next person: start screen sharing now	Lim
`CLIP n`	Live step n failed; play the recording of step n	Pan or Yorik
`EXPIRY OK / REFUND OK / REFUND LATE`	State of the 16:05 expiry bloc (16:10 and 16:50)	Yorik

1 Project Introduction & Project Management Summary (3+0 minutes)

Clock	16:30:00 – 16:33:00 (3 min talk + 0 min Q&A; talk ends 16:33:00)
Owner	Lim Jiechao
Slides	1, 2, 3, 4, 5, 6, 7, 8

Lecturer's presentation sequence · Presentation Guidelines · Briefing deliverables (V1.0)
1a — Explain the context and the business problem that the solution as a whole is trying to achieve.
1b — Explain the scope of the project.
1c — Explain the current status of the project including outstanding issues.
1d — Describe the actual project milestones and the total effort expended by each team member.
1e — Explain how many sprints planned, Definition of Done (DoD) achieved.
1f — Sprint retrospective & reflection in the next sprint
1g — Agile process artifacts like product backlog, user stories, sprint plans, burndown charts, sprint reviews and sprint retrospectives.

Tip
Lim opens at 16:30:00 sharp, with the deck shared and the notes window off the shared screen. Eight slides in three minutes: the tables carry the facts, the wording carries only the point of each slide.

Slide 1 BlocBuy

16:30:00–16:30:05 · 5 s

Speaker Lim · covers guideline item **1a**

Say *Good afternoon. We're Team 8, and this is BlocBuy.*

≈ 9 words · ≈ 4 s at 130 wpm · slot 5 s

Cues

- 16:30:00 — Lim opens swe5006-presentation.html in Chrome, presses S for the notes window (keep it off the shared screen) and presses T in either window to start the talk timer. In Zoom, share only the deck window.
- Backchannel: the team chat stays open. Lim repeats any question asked during the talk in one line and names who answers; the buffer is 17:04–17:10 (rubric G3).
- The roadmap line on the slide is the five sections of the lecturer's sequence; do not read it.

Hand-over *"The problem first."*

Slide 2 Group buying fails when buyers cannot commit together

16:30:05–16:30:30 · 25 s

Speaker Lim · covers guideline item **1a**

Say *Small merchants can only discount against committed volume, and a single buyer never reaches a bulk price. Every listing has two prices. You pay to start a team and share the link; friends join by paying too. If it fills in time, everyone gets the team price; if not, everyone is refunded automatically.*

≈ 53 words · ≈ 24 s at 130 wpm · slot 25 s · if short of time: If over time, drop "and share the link".

Cues

- Source: proposal §4 (27 Aug): the pay-first mechanic, two prices, automatic refund on expiry, modelled on the early Pinduoduo playbook; §6.1: Stripe test mode only.
- The one-sentence problem is the slide's headline: say it, then the mechanism. What makes it technical is that money is committed before the team exists, so every later section is about money safety under concurrency and failure.

Hand-over *"Here is what we set out to build."*

Slide 3 Scope: nine use cases, six quality attributes with targets

16:30:30–16:31:00 · 30 s

Speaker Lim · covers guideline item **1b**

Say *Scope is the nine use cases of our proposal, from browsing to admin listings, plus in-app notifications for SWE5001. Out of scope stays as proposed: no merchant self-service, no email or push, no live payments. On the right are the six quality attributes we committed to, with their targets. Each is tagged with where we show it; the system demo proves the first three.*

≈ 64 words · ≈ 30 s at 130 wpm · slot 30 s · if short of time: If over time, drop "from browsing to admin listings".

Cues

- In scope: proposal §6.1, UC1–UC9. Receive notifications (#215, #446) was added in Sprint 4 for SWE5001; it is off on the EC2 dev host by decision (backend ADR 0007, 11 Oct).
- Out of scope: proposal §6.2 verbatim list (merchant self-service GUI; analytics; recommendation; automatic team matching; notifications beyond in-app; native apps; live payments). We went beyond the first item for admins only: an admin console (frontend ADR 0015), shown on slide 4.
- Quality attributes and targets: proposal §6.3, quoted. "Shown in" maps each to the rubric rows that demonstrate it: 5.2 (correctness, security), 5.3 (performance, k6 on the SWE5001 ECS environment), 5.4 and §2 (maintainability), §3–§4 (code quality, auditability).

Hand-over *"How much of that did we deliver?"*

Slide 4 Promised, delivered, changed, dropped: status on 21 October

16:31:00–16:31:30 · 30 s

Speaker Lim · covers guideline item **1c**

Say *Against the proposal, every use case is in the code, and the admin console goes beyond it. The open issue is the money path on dev: Checkout returns, but the webhook never confirms, issues 517 and 223, our go/no-go on Friday.* **[click]** *Eight things changed, each with its record or its missing ADR; SWE5006 production was dropped. The open gaps are at the bottom.*

≈ 63 words · ≈ 29 s at 130 wpm · slot 30 s · if short of time: If over time, drop "and the admin console goes beyond it".

Cues

- Use cases: #24–#32 all closed completed 19–28 Sep; UC8 shipped (#31, #179, #181), so it was not sacrificed; UC9 admin console: frontend ADR 0015, admin stacks #294–#338.
- Money path on dev: #517 (16 Oct, Stripe reports success with test card 4242, receipt stays "Waiting for payment confirmation") and #223 (Checkout, scheduling and expiry refunds on dev), both open on 21 Oct, owner Yorik. Go/no-go Fri 23 Oct 18:00 (open item 1). `[VERIFY: re-check #517 and #223 on 25 Oct]`
- DevSecOps row (proposal §5.2, §6.4): OWASP Dependency-Check promised as a blocking check is not used; Trivy deps (Maven, pnpm) and Trivy config (IaC, Docker) run on PRs but are not required checks (ruleset 22734681 requires build-quality, SonarCloud Backend, docker, gitleaks, frontend-check, SonarCloud Frontend); the three plan/deploy/apply roles became three per-job roles (ECR push, backend deploy, frontend deploy); “no static credentials anywhere” is not met: two IAM users for SSM port forwarding, one with an active key (docs/deploy/aws-account-governance/README.md). No ADR for any of these (open item 7).
- Changed: team lifecycle State → DealStatus.canTransitionTo transition table (backend/docs/design/paid-group-participation.md; State is used for listings, slide 22). JWT → Sa-Token session in Redis with an HttpOnly cookie (backend-skeleton-plan.md D6/D7; frontend ADR 0002; no backend ADR, open item 7). JPA → MyBatis behind repository ports (skeleton plan §3 stack; no ADR, open item 7). ECS Fargate + RDS for dev → one EC2 host with Docker Compose and PostgreSQL on the host (backend ADR 0008). Secrets Manager → SSM SecureString and a root-only host env file (docs/deploy/backend-ec2-pipeline-operator-guide.md §7). UC9 "listing administration is API-only CRUD plus DB seeding" (proposal §6.1, §6.2: no merchant self-service GUI) → an admin console for admins (frontend ADR 0015, PRs #305–#307, 30 Sep). Proposal §6.4 patterns: Adapter for the Stripe payment abstraction → gateway ports under the external-interface transition strategy (slide 10; the GoF patterns are on slides 17–22); Observer for team resolution → kept, presented on UC5 → UC6 (slide 18). No ADR for the pattern changes (open item 7).
- Dropped for SWE5006: the prod release with a human gate and the ECS circuit breaker (#226 closed not planned 11 Oct; both exist for SWE5001 on ECS). Partial: Playwright e2e runs on API stubs, the signed-in suite on dev (#229) was not planned; burndown data is captured daily (board-snapshot.yml since 17 Sep) but no chart is generated; saved-plan Terraform promotion is a manual operator step only. Delivered: Checkstyle and both SonarCloud gates as required checks.
- Open gaps: port 22 open to 0.0.0.0/0 on the dev host (AWS read-only, 21 Oct); CodeQL off (ENABLE_CODEQL=false); CloudFront → ALB over HTTP accepted until 16 Nov (#227, NFR-37); dev EBS volume unencrypted (AWS, 19 Oct).

Hand-over *"How we got here, in milestones and hours."*

Slide 5 Milestones achieved and effort per member

16:31:30–16:31:45 · 15 s

Speaker Lim · covers guideline item **1d**

Say *Milestones left: the pipeline on dev in week one, the web app on 22 September, a tag per sprint. Right: hours per member, planned, because nobody logged actuals.*

≈ 28 words · ≈ 13 s at 130 wpm · slot 15 s · if short of time: If over time, drop "a tag per sprint".

Cues

- Milestones: proposal header (27 Aug); Sprint 1 report (dev pipeline live 5 Sep); stripe-sandbox-acceptance-20260920.md (local Sandbox acceptance 20 Sep); #93 comment (web app on dev 22 Sep); tags v0.1.0-sprint1/2/3 (14 Sep, 27 Sep, 11 Oct; git tag); admin console PRs #305–#307 merged 30 Sep (gh pr view, 22 Oct). Not milestones, so not listed: the 30 Sep local performance baseline and PR #424 (13 Oct).
- Planned hours: Sprint 2 plan docx (30 h each), sprint-03/plan.md §4 (30 h each), sprint-04/plan.md §5 (Liu Jin 51, Yorik 48, Eric 46, Pan 41.5, Lim 38). Open PR #538 rebalances Sprint 4 to Liu Jin 66, Eric 26, Pan 46.5 (open on 21 Oct). Sprint 1 hours were not recorded (Sprint 1 report contribution table left TODO). Team done hours: Sprint 2 134 of 150 (Sprint 2 report), Sprint 3 117 of 150 (sprint-03/review.md).
- Rubric 1.4: effort figures must match the fortnightly progress reports already submitted. Those reports carry no hours per member (Sprint 1 TODO, Sprint 2 activities and PR counts), so never present the planned hours as actuals.
  `[CONFIRM: actual effort per member agreed by all five, reconciled with the submitted progress reports]`

Hand-over *"Who owned what."*

Slide 6 Ownership: one use case and one design problem each

16:31:45–16:32:00 · 15 s

Speaker Lim · covers guideline item **1d**

Say *Each of us owns one use case and one design problem, and presents that problem in the next section. The last column shows what moved from the proposal's allocation.*

≈ 29 words · ≈ 13 s at 130 wpm · slot 15 s

Cues

- Ownership: issue #463 and sprint-04/plan.md §11 for the use cases; design problems as decided on 21 Oct (Strategy for UC1 pricing assigned to Lim, Template Method to Pan, Observer to Yorik). Proposal §6.1 indicative allocation: UC1 Strategy Pan; UC2 Pan; UC3 Factory Method and UC4 Eric; UC5 State/Observer and UC9 Liu Jin; UC6 Adapter Yorik; UC7 Observer and UC8 Jiechao.
- Owner strip (all nine use cases, rubric 1.5): UC3, UC4 Eric; UC6 Yorik; UC8 Lim; UC9 Liu Jin; notifications Pan, from #463 and sprint-04/plan.md §11. Proposed, not agreed (open decision 4, open item 21): UC1 Lim (owner of its Strategy problem; proposal had Pan), UC2 Pan (as in the proposal), UC5 Eric (the team statechart and its code are in his UC4 design; proposal had Liu Jin), UC7 Lim (built the UC7 frontend; proposal had Jiechao, i.e. Lim). Pan's use case, notifications, is a SWE5001 addition and off on dev; if the team prefers an in-scope one, UC2 is his.
  `[CONFIRM: use case owners for UC1, UC2, UC5, UC7 (open decision 4)]`
- Also on the matrix: the team-formation statechart (UC4–UC6, slide 16) is presented by Eric; the code behind it (DealStatus, GroupDeal guards) is in his UC3/UC4 design.
  `[VERIFY: UC9 attribution: #32 and backend design docs name Eric as UC9 implementer (PR #72); #463 gives UC9 to Liu Jin (open item 12)]`
- Report sections: RT 3.4.N (use case by member) and 3.5.N (design problem by member).
  `[FILL: report section numbers once #463 fixes the chapter order]`

Hand-over *"And how each sprint went."*

Slide 7 Four sprints: the Definition of Done reached, and what each retro changed 16:32:00–16:32:45 · 45 s

Speaker Lim · covers guideline item **1e**

Say *We place each sprint on the Definition of Done ladder from the EPAT module: coded, tested, integrated, releasable, released. Sprint 1 put the backend on dev, but no frontend. Sprint 2 merged the money path, tested, but with Checkout off on dev: integrated, not releasable. Sprint 3 aimed for a refund on dev; our rule says a local run does not count, so: integrated.* **[click]** *Each retro changed the next sprint: no burndown led to daily board snapshots; a ten-thousand-line pull request led to a 1,500-line limit and a mid-sprint check.*

≈ 90 words · ≈ 42 s at 130 wpm · slot 45 s · if short of time: If over time, drop "but no frontend" and "a mid-sprint check".

Cues

- Sprint dates: the submitted progress reports win where they exist: Sprint 1 report 1–12 Sep (reporting period), Sprint 2 report 15–26 Sep (docs/sprint2-progress-report.typ L81, L118), the same period as the Sprint 2 burndown on slide 8. Sprints 3 and 4 have no submitted report, so they follow the calendar in docs/agile/README.md (28 Sep–9 Oct, 12–23 Oct); that calendar gives 2–12 Sep and 14–25 Sep for Sprints 1 and 2, one day off the reports.
- Ladder: EPAT 01 Agile p26 (coded → tested → integrated → releasable → released). Placing each sprint on it is our own reading of the evidence, not a course rating. Basis, the same for every sprint: the sprint goal's increment; integrated = merged to dev with all required checks green, releasable = demonstrated on deployed dev (sprint-03/plan.md §8 "on dev" rule), released = in use by real users (none: Stripe test mode).
- Sprint 1 (2–12 Sep, #49): backend pipeline live on dev 5 Sep with health check and rollback; frontend not started (Sprint 1 report). Sprint 2 (14–25 Sep, #93): 9 of 11 stories, 134 of 150 h; frontend live 22 Sep; Checkout and scheduling off on dev (Sprint 2 report). Sprint 3 (28 Sep–9 Oct, #232): 117 of 150 h; review records the goal as not met on dev (sprint-03/review.md); DoD rule "on dev means demonstrated on deployed dev" (sprint-03/plan.md §8). Sprint 4 (12–23 Oct, #465): `[FILL: DoD step as recorded at the 23 Oct Sprint 4 review]`
- Retro → change: Sprint 1 retro (Sprint 1 report: no burndown or story points) → dated day logs and the scheduled board snapshot to docs/agile/metrics/ from 17 Sep (Sprint 2 report, "Acting on the Sprint 1 retrospective"). Sprint 2 retro (Sprint 2 report: ≈ 10,000-line Stripe slice, cliff burndown) → sprint-03/plan.md §8: PRs under ~1,500 lines, remaining hours in the estimate field, mid-sprint check 2 Oct. Sprint 3 retro not written (#451): the only recorded action is sprint-04/plan.md §7 (owners update the estimate field; the mid-sprint check verifies it). Sprint 4 retro due 23 Oct. `[FILL: Sprint 3 and Sprint 4 retro rows (open items 5, 6)]`
- One or two improvements per sprint follows the EPAT retrospective template (SCRUM_Process_Team01.xlsx, "Focus on 1 - 2 improvements").

Hand-over *"Two artefacts from that process."*

Slide 8 Agile artefacts: a story card, a burndown, a practice

16:32:45–16:33:00 · 15 s

Speaker Lim · covers guideline item **1g**

Say *From our GitHub board: one story card with its acceptance criteria, Sprint 2's burndown, flat then a cliff, and a refactoring with test-first evidence. The full set is in the appendix.*

≈ 31 words · ≈ 14 s at 130 wpm · slot 15 s

Cues

- Story card: #221 UC9-ext (US25), owner Liu Jin, six acceptance criteria ticked, merged in PR #242 (29 Sep), design problem field names State and Optimistic Offline Lock.
- Burndown: docs/sprint2-progress-report.typ, "Sprint Burndown Chart" (reconstructed from merge dates because the estimate field was empty); daily data docs/agile/metrics/*.csv since 17 Sep.
- Practice (EPAT 05 refactoring, EPAT 03 TDD): two red tests reproduced the below-one-count quote defect before the fix (docs/agile/sprint-03/log/2026-10-07.md, GroupDealPurchaseSupportImplTest.aTeamWithoutItsPaidLeaderCannotBeQuoted), then PR #424 (13 Oct) refactored admission into shared guards and a constructor-injected Factory Method (#421, #422).
- Full set (rubric 1.8: product backlog, user stories, sprint plans, burndowns, reviews, retros): backup B1, burndowns B2.

Hand-over *"Eric takes you through the design."*

Watch out

Never present planned hours as actual effort (slide 5), and never say payments or refunds work on dev unless the 24 Oct dry run proved it (slide 4). `[CONFIRM: slide 4 wording after the 23 Oct go/no-go]`

Q&A (during the talk / 17:04–17:10)

Likely question	Answer: decision → reason → evidence or limitation (30–45 s)	Owner
Why is Sprint 3 only “integrated” when the code was merged?	Our Definition of Done says "on dev" means shown on the deployed environment; a local run does not count. Checkout and scheduling were still off on dev at the freeze (#223), so the review recorded the goal as not met and carried it into Sprint 4. Evidence: sprint-03/review.md, sprint-03/plan.md §8.	Lim
Are these hours actuals?	No. We planned in hours per story; done hours are planned hours of finished work (Sprint 2 report, Sprint 3 review), and the Sprint 4 burndown sums planned estimates of cards not yet Done. Actual hours per person were not logged; that is a limitation we state.	Lim
Was the effort balanced across the five members?	The plans gave everyone 30 h in Sprints 2 and 3; Sprint 4 started at 38–51 h each and is being rebalanced (PR #538). Board cards differ in size, for example one Stripe pull request was about 10,000 lines, so card counts are not effort. `[CONFIRM: the effort split all five agree on]`	Lim
The proposal promised JWT. Where is that change recorded?	In the locked backend decisions of 30 Aug (backend-skeleton-plan.md D6/D7: Sa-Token session in Redis, RBAC from SQL tables) and frontend ADR 0002 (same-origin /api, HttpOnly cookie). Reason: a server session can be revoked and keeps the token out of JavaScript. Limitation: there is no numbered backend ADR for it (open item 7).	Lim → Liu Jin
Why did you build an admin GUI when the proposal excluded it?	The exclusion was a merchant self-service GUI. Admins needed to configure team terms on real listings to make START possible on dev (US25, #221), and an API-only flow was error-prone in the demo; the console reuses the same versioned API (frontend ADR 0015). It is extra scope, not a replacement.	Lim
What is your biggest open risk for the demo?	The Stripe webhook on dev (#517): payments stay waiting, so paid START, JOIN and refunds do not complete on dev yet. Go/no-go for the live payment steps is 23 Oct, with backup recordings by 24 Oct.	Lim → Yorik

2 Analysis & Design and Design Patterns (6+0 minutes)

Clock	16:33:00 – 16:39:00 (6 min talk + 0 min Q&A; talk ends 16:39:00)
Owner	Wang Ziang (Eric) for 2.1–2.4 and his design problem; then Yu Qianshuo (Yorik), Panpakorn Siripanich (Pan) and Lim Jiechao, each on their own design problem; Liu Jin for the full-form problem and 2.6
Slides	9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23

Lecturer's presentation sequence · Presentation Guidelines · Briefing deliverables (V1.0)

2a — Software architectural decisions that influence the logical architecture.
2b — Transition from analysis to design
2c — Use case diagram
2d — Analysis and design models
2e — Design problems and patterns
2f — Database schemas

Tip

Eric presents 2.1–2.4 and his own design problem; then Yorik, Pan and Lim each have one 20-second slide on their own problem; Liu Jin closes with the full-form State problem and the schema, and carries straight on into §3. The next speaker unmutes during the previous speaker's last sentence.

Slide 9 Logical architecture: COLA layers across six bounded contexts

16:33:00–16:33:45 · 45 s

Speaker Eric · covers guideline item **2a**

Say *I'm Eric. Our logical view as a UML package diagram: a modular monolith in four layers, each bounded context a package in every layer.* **[click]** *Controllers return one Result envelope against the committed OpenAPI contract.* **[click]** *Application services run the use cases.* **[click]** *The domain depends on nothing; infrastructure implements its ports, and eighteen ArchUnit rules enforce it.*

≈ 55 words · ≈ 25 s at 130 wpm · slot 45 s · if short of time: If over time, drop "Application services run the use cases."

Cues

- Diagram: src/diagrams/logical-layers.svg: backend Maven modules blocbuy-client, -adapter, -application, -domain, -infrastructure, -bootstrap (backend/pom.xml); contexts identity, catalog, groupbuy, order, payment, notification plus common (backend-skeleton-plan.md D1, D4). Invites and credits live inside groupbuy; "share" is only a frontend feature folder. 14 @RestController classes (InternalUserController only in the distributed topology).
- ArchUnit: grep -c @ArchTest backend/blocbuy-bootstrap/src/test/java/com/blocbuy/architecture/ArchitectureTest.java = 18 (22 Oct), e.g. domain_is_framework_free, application_uses_ports_not_tech, controllers_never_touch_persistence, no_field_injection, controllers_are_validated.
- Two domain ports are implemented in application, not infrastructure: PaymentBusinessPort (OrderFulfilmentAdapter) and PaymentIntentPort (PurchaseIntentAppService); both are cross-context seams inside commerce.
- Decisions: backend-skeleton-plan.md D1–D20 (30 Aug, pre-ADR locked decisions); backend ADR 0002, 0007 (still marked Proposed in its file although it runs on ECS), 0008; frontend ADR 0001, 0003. Full index: backup B3. Physical view: slide 29.
- Stack versions: Spring Boot 4.1.1, PostgreSQL 18, Redis 8, Sa-Token 1.46; React 19.3.0, TanStack Router 1.170.44, Query 5.104.2, TypeScript 7.0.2 (frontend/package.json, 22 Oct).

Hand-over *"The rules that turned analysis into this design come next."*

Slide 10 Transition strategies from analysis to design

16:33:45–16:34:25 · 40 s

Speaker Eric · covers guideline item **2b**

Say *Our transition strategies, in SAD 04a's shape: per design concern, the analysis object types it changes and what they become. GUI boundaries become routes and query hooks; controls become REST controllers behind a generated stub;* **[click]** *entities get repository ports.* **[click]** *Four have no ADR yet.*

≈ 44 words · ≈ 20 s at 130 wpm · slot 40 s · if short of time: If over time, drop the three examples (they stay in the table).

Cues

- Template: SAD 04a p36–37 (a strategy deals with one design concern, usually an NFR; impacted object types, use cases, static and dynamic changes). p36 names "concurrency of user requests" as a strategy subject, which is why concurrency is a row.
- Records: backend-skeleton-plan.md D1–D20 (language, persistence D1/D9, distribution D11/D12, security D6/D7); backend ADR 0001 (purchase intent, group terms; lock and unique index), 0002 (expiry transition and events), 0003 (credit after commit), 0004 and 0005 (Stripe path and its gate), 0007 (outbox); frontend ADR 0001 (stack), 0002 (cookie session), 0003 (generated client), 0009 (envelope).
- No ADR: language and framework, persistence (MyBatis, while the proposal said JPA), the scheduling job design, the concurrency approach as such (open item 7). The design docs hold them: backend/docs/design/paid-group-participation.md (UC4 last-seat race, L283–352).
- Security in code: SaTokenConfig global interceptor on /api/v1/** with public paths; StpUtil.checkRole(ADMIN) on admin controllers; X-BlocBuy-Request on money writes (StripeCheckoutController.checkCsrf); RedisRateLimitGateway. Scheduling: TeamExpirySweepJob fixedDelay PT1M under LeasedJobRunner (sys_scheduler_lock).

Hand-over *"Here is what one of them does to a use case."*

Slide 11 One strategy pair: analysis objects become design classes

16:34:25–16:35:00 · 35 s

Speaker Eric · covers guideline item **2b**

Say *One pair, from UC4.* **[click]** *Before: slide 13's analysis classes, with no platform.* **[click]** *The key says what each strategy makes of each type.* **[click]** *Persistence: the Team becomes an aggregate behind a repository port that MyBatis implements. In sequences, control calls entity becomes lock, aggregate, guarded update.*

≈ 45 words · ≈ 21 s at 130 wpm · slot 35 s · if short of time: If over time, drop the last sentence (it stays on the slide).

Cues

- Diagram: src/diagrams/transition-pair.svg; crops 1 (analysis side: JoinTeamUI, JoinTeamControl, Team, PaymentGateway, Payment, the same names and operations as uc4-analysis-class.svg on slide 13, cut to five classes), 2 (strategy key and analysis → design mapping), 9 (PaidGroupDealRepository ◁ PaidGroupDealRepositoryImpl, persistence dependency inversion). The full design side (17 tagged classes) is the overview; slides 14–15 show the UC4 design model itself. Report §3.2 (transition strategies).
- Analysis names (JoinTeamUI, JoinTeamControl, PaymentGateway) are analysis-model names, not code: SAD 03a p21 names the control after the use case.
- Design classes: products.$productId.tsx + use-checkout.ts (Pres); generated client from contracts/openapi.json (Dist, F-ADR 0003); StripeCheckoutController → PaymentBrowserService → StripeCheckoutService (Dist, Sec); GroupDeal + PaidGroupDealRepository (domain) ◁ PaidGroupDealRepositoryImpl + PaidGroupDealMapper.xml + BizGroupDealDO + GroupDealConverter (Pers); CheckoutGateway ◁ StripeCheckoutGateway (Ext). Port method names: reserveAndLock, lockDeal, insertJoinedMember.
- Sequence change: SAD 04a p68–69 shows the same move for its View Car example (controller → DAO with alt [found]); ours goes through a repository port, which the persistence strategy makes one per aggregate root (SAD 04a p75).

Hand-over *"Those use cases, in one diagram."*

Slide 12 Use case diagram: in scope, out of scope and status

16:35:00–16:35:30 · 30 s

Speaker Eric · covers guideline item **2c**

Say *Our use case diagram.* **[click]** *In scope: start and join include paying upfront; invite attribution extends join.* **[click]** *The Scheduler resolves teams; refund extends resolution on expiry; Stripe is secondary.* **[click]** *Out of scope, greyed, as the proposal excluded. Badges: the money path is coded, not yet on dev.*

≈ 46 words · ≈ 21 s at 130 wpm · slot 30 s · if short of time: If over time, drop "Stripe is secondary".

Cues

- Diagram: src/diagrams/use-case.svg; crops A (Buyer, money path), B (resolution: UC5, UC6 extend, Scheduler, Stripe), C (out-of-scope package). Use case names: proposal §6.1; out of scope: proposal §6.2 (merchant self-service GUI, analytics, recommendations, automatic team matching, email/SMS/push, native apps, live payments).
- Notation: «include» arrow from base to included, «extend» from extending to base (SAD 01b p41–42); a time-driven Scheduler actor as in SAD 03c p5. Google is a secondary actor of UC2.
- Status badges are our own addition (team ruling): UC1, UC2, UC7, UC8, UC9 on dev; UC3–UC6 coded, but the paid flow does not complete on dev (#517 webhook, #223 scheduling and refunds). Receive notifications: coded, off on dev (backend ADR 0007 scope decision, 11 Oct). UC7 credit fires only once paid joins work.

Hand-over *"UC4, join a team, from analysis to design."*

Slide 13 UC4 join: the analysis model

16:35:30–16:35:50 · 20 s

Speaker Eric · covers guideline item **2d**

Say *UC4, analysis first: boundary, control and entity only, no platform.* **[click]** *Attributes sit only on entities: the Team holds its size and deadline.* **[click]** *Joining creates a membership if the window is open and a seat is free;* **[click]** *otherwise the payment is refunded.*

≈ 41 words · ≈ 19 s at 130 wpm · slot 20 s

Cues

- Diagrams: src/diagrams/uc4-analysis-class.svg (crop 2: Team, Membership, Payment with multiplicities) and uc4-analysis-seq.svg (crops 3 and 4: admit, alt [window open and seat free] «create» Membership / [else] «create» Refund). Conventions: SAD 03a p8 (three stereotypes), p29 (attributes only on entities), p49–50 (actors talk to boundaries only).
- The pre-payment check (lockJoinTerms in code) is drawn as checkAdmission with a break [not admissible] fragment. Order is associated only with Team in the analysis class diagram, for readability; the code also links order to payment and member to order.
- Full UC4 models in the report: §3.4 (Eric).

Hand-over *"And the design model."*

Slide 14 UC4 join: the design class diagram

16:35:50–16:36:05 · 15 s

Speaker Eric · covers guideline item **2d**

Say *Each design class carries the tag of the strategy that made it.* **[click]** *Payment reaches groupbuy only through a port;* **[click]** *the Team is an aggregate behind a repository port,* **[click]** *implemented with MyBatis.*

≈ 31 words · ≈ 14 s at 130 wpm · slot 15 s · if short of time: If over time, drop "implemented with MyBatis".

Cues

- Diagram: src/diagrams/uc4-design-class.svg (crops 5: PaymentBusinessPort ◁ OrderFulfilmentAdapter, the context seam; 7: GroupDeal and «interface» PaidGroupDealRepository; 8: PaidGroupDealRepositoryImpl, PaidGroupDealMapper, BizGroupDealDO). Every class and operation carries file:line in the diagram spec; the complete diagram is backup B14.
- Elisions: PaymentBusinessPort declares five operations (lockQuote, lockForSettlement, fulfil, fulfilmentEvidence, lockExpiredTeam; PaymentBusinessPort.java:13–38); the diagram shows the two the join uses. Two domain ports are implemented in application, not infrastructure: PaymentBusinessPort (OrderFulfilmentAdapter) and PaymentIntentPort (PurchaseIntentAppService).
- Notation: typed signatures, visibility, multiplicities (SAD 01b p14–16); strategy tags trace each design class to slide 10 (SAD 04a p37; RT 3.4.1 "the design complies with the applicable transition strategies"). Report §3.4 (Eric).

Hand-over *"And the sequence that runs through it."*

Slide 15 UC4 join: the design sequence, paid join by webhook

16:36:05–16:36:20 · 15 s

Speaker Eric · covers guideline item **2d**

Say *After the webhook,* **[click]** *one transaction locks the team row first: a racing join waits;* **[click]** *the update must see the size it read; the last seat publishes TeamSucceeded;* **[click]** *a loser is owed a refund.*

≈ 33 words · ≈ 15 s at 130 wpm · slot 15 s · if short of time: If over time, drop "so a racing join waits".

Cues

- Diagram: src/diagrams/uc4-design-seq.svg; each crop repeats the lifeline header strip (top 180 px). Crops: 3 (lockForSettlement → lockDeal FOR UPDATE), 5 (guarded UPDATE, MemberJoined, opt TeamSucceeded), 7 (red: BizException, compensate, receipt compensation_required). Every message carries file:line in the spec; the whole sequence is backup B14.
- Lock order on the webhook path: deal (StripeCheckoutService.lockForSettlement → OrderFulfilmentAdapter → settlement.lockTeam → lockDeal FOR UPDATE), then intent, then payment (FOR UPDATE), then the receipt; joinConfirmed locks the deal again inside the sam transaction (already held). Guard order in GroupDeal.joinPaid: price → new member → windo → seat; the first failure decides the code (PAYMENT_MISMATCH, ALREADY_MEMBER, DEAL_CLOSED, DEAL_FULL).
- Guarded update: PaidGroupDealMapper.xml updateDeal ... WHERE current_size = #{expectedSize} AND status = #{expectedStatus}; biz_group_deal.version is incremented but compared (the payment save does compare version: PaymentMapper.xml). Backstops: uq_biz_group_member_deal_user, uq_biz_group_member_payment_id, uq_biz_order_payment_id.
- The critical fragment marks the single database transaction (READ COMMITTED, 10 s timeo UML has no transaction fragment, so we use a critical region and say so. Loser path: receipt compensation_required → requireRefund → biz_refund. Evidence: StripeLocalIntegrationTest.twentyBuyersRaceForLastSeatAndNineteenRefundObligationsPe (local). Report §3.4 (Eric).

Hand-over *"The team state behind those guards."*

Slide 16 Team-formation statechart

16:36:20–16:36:30 · 10 s

Speaker Eric · covers guideline item **2d**

Say *The team's statechart.* **[click]** *OPEN takes paid joins while seats remain;* **[click]** *the last seat succeeds it, the deadline expires it.*

≈ 19 words · ≈ 9 s at 130 wpm · slot 10 s

Cues

- Diagram: src/diagrams/team-statechart.svg, consistent with DealStatus.canTransitionTo (this == OPEN && target != null && target != OPEN) and the GroupDeal guards. Time event at(expireTime) because the deadline is an absolute time from the terms, not a duration after entering OPEN; TeamExpirySweepJob detects it every minute.
- Why drawn: SAD 03c p4 and p11 call for a statechart when an object's state affects more than one use case; team state gates UC4, UC5, UC6 and UC8. The proposal also promised the state machine (§6.4).
- FAILED and CANCELED are declared in DealStatus but never written (grep, 21 Oct); shown greyed so nobody reading the enum is surprised.
- UML 2.5 details: the initial transition carries only its effect (a confirmed paid START creates the team, in a note), the three paidJoinConfirmed transitions out of OPEN have complete, disjoint guards (admissible and not last seat, admissible and last seat, ¬admissible), and no final state is drawn because SUCCEEDED and EXPIRED still answer late paid joins with compensation_required.

Hand-over (Eric continues) My own design problem.

Slide 17 UC3 start a team: Factory Method

16:36:30–16:36:50 · 20 s

Speaker Eric · covers guideline item **2e**

Say *My problem: the start rules stay fixed while the kind of team may vary.* **[click]** *Factory Method: the service sees only the Creator, whose final initiate validates;* **[click]** *a ConcreteCreator's createGroupDeal builds the Product,* **[click]** *so every creator keeps the rules. Liability: one creator, one team class.*

≈ 44 words · ≈ 20 s at 130 wpm · slot 20 s

Cues

- History: the Creator hierarchy exists since b6f7fafd (paid participation). Until PR #424 (merged 13 Oct, 081047f5) the Client still named the ConcreteCreator: `private final PaidGroupDealCreator creator = new StandardPaidGroupDealCreator();` (git show 081047f5^); #424 made it constructor-injected and added GroupDealCreationConfig (#421), so the Client depends only on the abstract Creator. Cause for redesign (SDP 05a p15): creating an object by specifying a class explicitly; without the pattern the service would call GroupDeal.startPaid itself.
- Polymorphic Factory Method: PaidGroupDealCreator is abstract with a public final initiate and a protected abstract createGroupDeal; StandardPaidGroupDealCreator overrides it. SDP 01a p32–35 presents the parameterised create() as a Factory Method variant and notes on p34 that making it static "forbids subclassing of the base factory class"; ours is the subclassing form. Calling a static create(type) a simple factory rather than GoF Factory Method is our team ruling (rubric 2.5), not the deck's wording. GroupDeal.startPaid is package-private and static, and is not claimed as a pattern.
- What varies (SDP 05a p13, GoF p30): "subclass of object that is instantiated". The pattern provides that variation point: a second ConcreteCreator decides which team class initiate builds, while initiate keeps the start rules (GoF p136: a template method calling a factory method). Needed because: proposal §6.3 makes extensibility the primary design driver (new team-resolution behaviour without editing domain logic). Liability, honestly: the flexibility is not exercised yet; GroupDeal is public final (GroupDeal.java:19) and StandardPaidGroupDealCreator is the only, final, ConcreteCreator, chosen in GroupDealCreationConfig. A second team type would need GroupDeal opened up or a new product class.
- Implementing classes and tests (G10, IDE open): PaidGroupDealCreator, StandardPaidGroupDealCreator, GroupDeal, PaidGroupDealService, GroupDealCreationConfig; PaidGroupDealServiceTest.startUsesTheInjectedCreator, PaidGroupDealTest.createsOpenDealWithExactlyOnePaidLeader, initiationRejectsExpiredTermsInvalidSizeAndMissingClock.
- Diagram: src/diagrams/fm-start.svg (crop A1). Full write-up: backend/docs/design/paid-group-participation.md L108–262; report §3.5 (Eric).

Hand-over *"Yorik, what happens when a team ends."*

Slide 18 UC5 to UC6 team outcome: Observer

16:36:50–16:37:10 · 20 s

Speaker Yorik · covers guideline item **2e**

Say *I'm Yorik. An expired team must cancel orders and record refunds; calling them directly is tight coupling.* **[click]** *Observer: Spring's publisher is the Subject, adapting each subscriber method as an Observer,* **[click]** *run before commit, in order. New consequence, new subscriber.*

≈ 39 words · ≈ 18 s at 130 wpm · slot 20 s

Cues

- Participants: Spring’s ApplicationEventPublisher (notify) and ApplicationEventMulticaster (at observer list) play the Subject, like GoF’s ChangeManager; Observer = ApplicationListener; ConcreteObserver = Spring’s TransactionalApplicationListenerMethodAdapter, which TransactionalEventListenerFactory creates for each @TransactionalEventListener method ar which realises ApplicationListener (spring-tx 7.0.9); our subscriber classes implement no in (OrderLifecycleSubscriber.java:32, TeamExpiryRefundSubscriber.java:26), the adapter invol their annotated methods; GroupDealExpiryService (and PaidGroupDealService for TeamSucceeded, MemberJoined) is the event source, a client of the Subject, not a ConcreteSubject: it holds no observer list; deal.expire(now) is a call on the GroupDeal aggre note in the sequence); adapted subscribers OrderLifecycleSubscriber @Order(10) and TeamExpiryRefundSubscriber @Order(20), both BEFORE_COMMIT; ShareCreditSubscriber handles MemberJoined AFTER_COMMIT (ADR 0003).
- Why before commit: ADR 0002, both writes commit with the expiry or neither does. Tests (G TeamExpiryRefundSubscriberTest.expiryRecordsRefundObligationsForThatTeam, aFailureToRecordObligationsEscapesAndRollsTheExpiryBack; OrderLifecycleSubscriberTest.expiryCancelsPendingOrdersStampedWithTheTeamsResolutic No test asserts the 10 → 20 order directly.
- Refund idempotency (durable obligation, provider idempotency key, lease and version fence mechanism of the external-interface strategy, not part of the GoF pattern: backup B6. Diagr src/diagrams/observer-expiry.svg (crop A2). Report §3.5 (Yorik).
- Liability: implicit control flow, and a wrong phase annotation is a silent bug (ADR 0002 calls i easiest thing here to get wrong").

Hand-over *"Pan, the notifications those events feed."*

Slide 19 Notifications: Template Method for outbox transports

16:37:10–16:37:30 · 20 s

Speaker Pan · covers guideline item **2e**

Say *I'm Pan. Our outbox relay publishes to SQS locally and SNS on ECS with the same batching; only two steps differ.* **[click]** *Template Method: a final publish runs the loop* **[click]** *and calls entry and send, which each transport implements.*

≈ 38 words · ≈ 18 s at 130 wpm · slot 20 s

Cues

- Participants: AbstractClass BatchingOutboxEventTransport<E> (package-private; public final publish = template method; protected abstract entry and send = primitive operations); ConcreteClasses SnsOutboxEventTransport, SqsOutboxEventTransport; client OutboxRelayService through the domain port OutboxEventTransport; one bean chosen by blocbuy.events.transport (OutboxTransportConfiguration).
- What varies: "steps of an algorithm" (SDP 05a p12–14); Strategy rejected because the whole algorithm does not vary. The brute force (two copies of the loop) is hypothetical: both classes were born in the PR #485 follow-up, with the shared batch publisher (ADR 0007 review amendment).
- Tests (G10): OutboxBatchFailureTest (6, parameterised over both providers: middleBatchExceptionPreservesSuccessesAndStillSendsLaterRows, leaseLossDuringFirstSendSkipsAllSubsequentBatches, ...), SnsOutboxEventTransportTest, SqsOutboxEventTransportTest. Runs locally and in CI against ElasticMQ; the relay is off on SWE5006 dev (ADR 0007 scope). Report §3.5 (Pan).

Hand-over *"Lim, pricing."*

Slide 20 UC1 pricing: Strategy

16:37:30–16:37:50 · 20 s

Speaker Lim · covers guideline item **2e**

Say *Mine is UC1 pricing: browse and the team quote price by the listing's pricing mode.* **[click]** *Strategy: both services get that mode's strategy from a resolver* **[click]** *and delegate the arithmetic. One strategy today; the proposal made new pricing schemes a design driver.*

≈ 41 words · ≈ 19 s at 130 wpm · slot 20 s

Cues

- Participants: Strategy PricingStrategy (domain, stateless); ConcreteStrategy DualPricingStrategy; Contexts ProductAppServiceImpl, TeamPriceAppServiceImpl; PricingStrategyResolver is a registry (not a GoF participant) built once from all PricingStrategy beans, stores Map.copyOf, fails startup on a duplicate mode and fails a request on an unknown one; key PricingMode (only DUAL), from biz_product.pricing_mode.
- Liability (SDP 05a p35, "apply only if the flexibility is needed"): one ConcreteStrategy today (backend/docs/design/team-price-quotes.md L55). Justification: proposal §6.3 names "new pricing schemes ... addable without modifying existing domain logic" as a primary design driver; a new mode still needs an enum constant, a class, a bean and tests.
- Tests (G10): PricingStrategyResolverTest.resolvesRegisteredStrategyByMode, duplicateModeFailsFastAtStartup, unknownModeIsBusinessErrorNotSilentFallback; DualPricingStrategyTest (9). Ownership: report section owner; the backend code was written by Liu Jin and Eric (team-price-quotes.md L32–33). Report §3.5 (Lim).

Hand-over *"Liu Jin, one problem in full."*

Slide 21 Design problem: listing rules that depend on its state

16:37:50–16:38:20 · 30 s

Speaker Liu Jin · covers guideline item **2e**

Say *I'm Liu Jin, with one problem in full. Publish, delist, revise and delete are legal only in some states of a listing.* **[click]** *A brute-force design repeats the status switch in all four operations, in red;* **[click]** *in the sequence, publish branches on status. What varies is the states of an object; we weighed State, Strategy and a transition table.*

≈ 58 words · ≈ 27 s at 130 wpm · slot 30 s

Cues

- Diagram: src/diagrams/state-before.svg, a hypothetical brute-force design (RT 3.5: "using brute force design if necessary"), not code that existed: it is the alternative backend/docs/design/admin-listings.md L67–68 names and rejects ("repeated status switches in every caller"). The rules inside the switches are the real ones (ProductStatus.java L12–74).
- Real operation names: Product.publish(), delist(), revise(details), requireDeletable(); client AdminListingAppServiceImpl; endpoint PUT /api/v1/admin/products/{productId}/status (AdminListingController).
- Cause for redesign: GoF State applicability ("operations have large, multipart conditional statements that depend on the object's state"); none of SDP 05a p15's causes lists State, so we do not force one. What varies: SDP 05a p13 "States of an object".
- Authorship: admin-listings.md names Eric as implementer and Liu Jin as reviewer; Liu Jin owns the report section (#463, open item 12).

Hand-over (Liu Jin continues) The design after State.

Slide 22 After: the State pattern, and why the team uses a table

16:38:20–16:38:45 · 25 s

Speaker Liu Jin · covers guideline item **2e**

Say *We chose State.* **[click]** *Product delegates to ProductStatus; each enum constant with a body compiles to its own subclass and overrides only the moves it allows; the rest refuse with 409.* **[click]** *Each ConcreteState answers publish its own way, with no switch. The team uses a table instead: only OPEN has behaviour.*

≈ 50 words · ≈ 23 s at 130 wpm · slot 25 s

Cues

- Diagram: src/diagrams/state-after.svg (crops A2, B); the after-sequence alt has one operand per kind of ConcreteState (override: DRAFT, OFF_SALE; inherited default: ON_SALE), chosen by dynamic dispatch, not by a switch; listing statechart DRAFT -publish→ ON_SALE -delist→ OFF_SALE -publish→ ON_SALE, delete only in DRAFT: backup B13.
- Enum constant bodies: ProductStatus DRAFT, ON_SALE and OFF_SALE all have bodies; javac compiles each to an anonymous subclass (ProductStatus$1 … $3), so it is one class per state, shared and stateless (GoF State implementation 2). Defaults throw BizException(PRODUCT_STATE_CONFLICT), HTTP 409.
- Product is immutable: transition returns a new revision with version + 1; updateListing(..., expectedVersion) returns 412 on a stale version (Optimistic Offline Lock is a persistence mechanism, not GoF). Liability: AdminListingAppServiceImpl.changeStatus still switches on the target to pick publish or delist; Product.isOnSale() compares status for the browse filter.
- Why not State for the team: DealStatus has one move (OPEN → terminal) and its consequences are events (slide 18), so a transition table suffices; State would add three classes with no behaviour (SDP 05a p35). This is the 1.3 delta on slide 4: the proposal promised State for the team lifecycle.
- Tests (G10): ProductStatusTest.onSalePublishThrowsStateConflict, offSaleDelistThrowsStateConflict, currencyOnlyChangeIsRejectedAfterPublication; ProductLifecycleTest (7). Report §3.5 (Liu Jin).

Hand-over (Liu Jin continues) And where all of this is stored.

Slide 23 Database schema: core entities and mapping decisions

16:38:45–16:39:00 · 15 s

Speaker Liu Jin · covers guideline item **2f**

Say *The core schema. Keys are UUIDv7; associations are logical foreign keys, checked by the application.* **[click]** *Unique indexes are the race backstops: one membership per buyer per team,* **[click]** *one refund per payment.*

≈ 31 words · ≈ 14 s at 130 wpm · slot 15 s

Cues

- Diagram: src/diagrams/core-erd.svg: biz_product, biz_group_deal, biz_group_member, biz_purchase_intent, biz_payment, biz_order, biz_refund and sys_user, from Flyway V1–V28 (backend/blocbuy-bootstrap/src/main/resources/db/migration). Crops 2 (deal, membership: uq_biz_group_member_deal_user) and 6 (payment, refund: biz_refund_payment_id_key, an inline UNIQUE on payment_id).
- No FK or CHECK constraints is the team's DDL convention (.claude/skills/pgsql-ddl, V5 header): integrity is application-managed. That is our rule, not the course's; SAD 04a p55 maps associations to FK columns, which we keep as logical columns.
- Other details if asked: biz_purchase_intent has no status column (state from fulfilled_payment_id and expired_at); biz_group_deal.version is incremented, not compared; uq_biz_payment_request_key carries the Checkout idempotency key. All 29 tables: backup B4; zoomable copy: open src/diagrams/full-erd.svg in a browser tab.

Hand-over *"Now, how we tested all this."*

Watch out

Handover allowance: speech is about 285 s of the 360 s slots, so the four handovers inside §2 (slides 18, 19, 20, 21) get 15–20 s each, rehearsed short: the next speaker unmutes and starts with their first line, no greeting beyond their name. Beyond that, segment 2 has no slack: answer design questions in one sentence and park the rest.

Q&A (during the talk / 17:04–17:10)

Likely question	Answer: decision → reason → evidence or limitation (30–45 s)	Owner
Why a monolith on dev and services on prod?	Decision: one codebase, two topologies (backend ADR 0008). Reason: SWE5006 grades a complete runnable system, so dev stays the monolith on one EC2 host; SWE5001 needs three services on ECS. The services are thin assemblies of the same modules, and ArchUnit holds the boundaries in the monolith too.	Eric
Is ProductStatus really the State pattern, or an enum switch?	State, with enum constants as the concrete states: each constant with a body compiles to its own subclass, overrides only the moves it allows and returns the next state; the defaults refuse with 409, and Product delegates without branching on status. Evidence: ProductStatus.java, ProductStatusTest. GroupDeal uses a transition table (DealStatus.canTransitionTo), which we do not call State.	Liu Jin
How do you stop a double refund?	A durable obligation, then an idempotent call. The obligation is unique per payment and commits with the expiry; the Stripe idempotency key is built from its ID; a lease plus a version fence stops two workers; after 23 hours the worker only searches Stripe, never creates. Evidence: RefundServiceTest.lostResponseRetriesSameDurableKey. Limitation: proven locally with a provider substitute, not on dev (#223).	Yorik
Why a fixed 30 s retry and not exponential backoff?	A fixed delay in RefundMapper.xml: the idempotency key makes each retry safe, and the 23-hour cutoff ends automatic retries. There is no exponential backoff or jitter, so we call it a fixed delay.	Yorik
Why a Factory Method with only one creator?	The creator is a seam: initiate is final, so any creator keeps the validation, and constructor injection (#421) lets a test or configuration supply another creator without editing the service. Limitation: one creator, chosen at build time; we do not claim runtime-pluggable creation.	Eric

Strategy with only one strategy: is that not speculative?	It is a liability we state (SDP 05a p35). The seam exists because proposal §6.3 made new pricing schemes a primary design driver; the selection is data-driven (biz_product.pricing_mode) and the two contexts never name the algorithm. A second mode would add a constant, a class, a bean and tests, and leave browse and quote untouched. We do not claim it has been exercised.	Lim
Why not the State pattern for the team, as the proposal said?	Behaviour differs per state only in OPEN: SUCCEEDED and EXPIRED are final and refuse everything with the same code. A transition table (DealStatus.canTransitionTo) plus the GroupDeal guards covers that; State would add three classes with no behaviour. We used State where operations differ in kind per state, on listings (slide 22), and record the change on slide 4.	Liu Jin → Eric
Is your repository or DAO a design pattern?	Not a GoF one, so it is not on the pattern slides. It is the persistence transition strategy (SAD 04a p64–65, the simple DAO; one per aggregate root, p75): a domain repository interface, implemented in infrastructure by a MyBatis mapper. The same holds for the outbox, idempotency keys and the Optimistic Offline Lock: mechanisms, not GoF patterns.	Eric
How do analysis and design differ in your UC4 models?	Analysis (slide 13) uses only boundary, control and entity, with attributes on entities and no platform (SAD 03a). Design (slide 14) has the real classes with typed signatures, each tagged with the transition strategy that produced it, and the sequence shows the transaction, the row locks and the guarded update. The full set is in report §3.4.	Eric
Is there a database per service?	On dev: one PostgreSQL database, schema "domain", all tables. On ECS: one RDS instance with a database per service, and each runtime role is granted only its own tables (ADR 0008). Full list: B4.	Liu Jin
Do notifications work on dev?	No, by decision: the dev host runs no broker and leaves the outbox writer, relay and consumer off (application-ec2.yml; ADR 0007 scope decision, 11 Oct). The path runs on ECS through SNS and SQS, and locally and in CI against ElasticMQ (OutboxRelayElasticMqIT, NotificationConsumerElasticMqIT).	Pan

3 Types of Testing Conducted (3+0 minutes)

Clock	16:39:00 – 16:42:00 (3 min talk + 0 min Q&A; talk ends 16:42:00)
Owner	Liu Jin
Slides	24, 25

Lecturer's presentation sequence · Presentation Guidelines · Briefing deliverables (V1.0)

3a — Types of Testing Conducted
3b — Well structured source code with unit test cases for classes with more complex behaviours.

Tip

Liu Jin continues from slide 23 without a handover.

Slide 24 Testing on the Agile Testing Quadrants

16:39:00–16:40:50 · 110 s

Speaker Liu Jin · covers guideline item **3a**

Say *We map our tests onto the Agile Testing Quadrants and tag each with its level. Quadrant one, technology-facing and supporting the team: 162 unit test classes, plus 23 that boot Spring or Testcontainers, 77 Vitest component files, 49 integration classes, 45 of them on real PostgreSQL, Redis and ElasticMQ in Testcontainers, the contract check and 18 ArchUnit rules. They target the complex classes we promised: the team state machine; the last seat, where twenty paid buyers race on real PostgreSQL and exactly one wins; and refund idempotency, where a lost Stripe response is retried with the same key. Two red tests caught a join-quote defect before its fix.* **[click]** *Quadrant two, business-facing: story-level acceptance tests through real HTTP and signed webhooks, and eight Playwright specs, run on the preview build and against dev after each deploy. That smoke test caught a 502 while the single host restarted.* **[click]** *Quadrant three is manual, and we keep it honest: a logged Stripe Sandbox run, locally, and cookie checks on dev; no defect came from them, and no outside users tested it.* **[click]** *Quadrant four uses tools: the load test on the next slide, and four kinds of security scanning, static analysis, dependencies, image and infrastructure configuration, and secrets, which Pan shows gating the pipeline. The weekly full-history secret scan still has four badge tokens open.*

≈ 220 words · ≈ 102 s at 130 wpm · slot 110 s · if short of time: If over time, drop "Two red tests caught a join-quote defect before its fix." (it stays on the slide).

Cues

- Model: EPAT 03 Test Driven Development v2.1 p7 (Agile Testing Quadrants: Q1 unit and component, automated; Q2 functional and story tests, automated and manual; Q3 explorator usability, UAT, manual; Q4 performance, load, security, tools) and p5 (levels: unit, integratior system). The level tag on each row is our classification. Placement: repository, SQL and contract integration tests are technology-facing, so Q1 at integration level; Q2 holds only story-level tests that exercise a user story end to end: StripeRolloutAcceptanceIT (8 tests: re HTTP endpoints, scheduler, Flyway schema, webhook signatures; only OAuth and Stripe network calls substituted) and the Playwright e2e specs.
- Counted in the repository on 22 Oct at a1c6ac6e: find backend -name "*Test.java" = 186 (+ ArchitectureTest), 1,329 @Test/@ParameterizedTest; of the 185, 23 boot @SpringBootTe or Testcontainers (grep -l "@SpringBootTest|Testcontainers|@Container|PostgreSQLContainer", 22 Oct at 68325744 so 162 are unit level; "*IT.java" = 49, 45 of them start Testcontainers (postgres:18, redis:8.1 ElasticMQ), 242 annotations; grep -c @ArchTest ArchitectureTest.java = 18; frontend/src "*.test.ts*" = 77; frontend/e2e = 8 specs, 24 test() declarations. Per module: backup B7.
- Complex classes (proposal §6.3 "unit tests concentrated on classes with complex behaviour state machine DealStatusTest, PaidGroupDealTest (terminalStatesRejectJoins, lastPaidMemberSucceedsWithoutMutatingOriginalSnapshot); capacity StripeLocalIntegrationTest.twentyBuyersRaceForLastSeatAndNineteenRefundObligationsPer (Spring Boot with production ports, PostgreSQL in Testcontainers; twenty paid JOINs release together; asserts 2 members, the leader and one joiner, and 19 refund obligations; a *Test b name, integration by level). StripeCheckoutFlowTest.concurrentPaidBuyersCannotBothTakeLastSeat is not cited: it races through a test fake of the join path; refund idempotency RefundServiceTest.lostResponseRetriesSameDurableKey.
- Caught: (Q1) sprint-03/log/2026-10-07.md, two zero/negative-count red tests reproduce the quote defect (GroupDealPurchaseSupportImplTest.aTeamWithoutItsPaidLeaderCannotBeQuoted; #422, fi in #424); ArchUnit controllers_are_validated required @Validated on InternalUserController (sprint-04/log/2026-10-13.md, #468). (Q2) ci-frontend runs 37102432667 (3 Oct) and

37182996807 (4 Oct): e2e-dev got 502 from /api/actuator/health inside deploy-existing-e of the same commit. (Q3) none logged from the 20 Sep Sandbox run or the 8 Oct cookie che (Q4) security.yml weekly full-history gitleaks: 24 matches, 20 SHA-256 digests in a contrac baseline re-hashed and fingerprinted (PR #580), 4 badge-token findings still open (slide 30 docs/quality/quality-gates.md).

- Where it runs: ci-backend.yml build-quality (mvnw verify: unit, ArchUnit, ITs; required), contract-check (export, git diff --exit-code contracts/, openapi-diff, not required); ci-frontend.yml frontend-check (Vitest; required), e2e-preview (PRs), e2e-dev (after deploy-dev). Latest local full backend verify: 2,048 passed, 1 opt-in performance skip (sprint-04/log/2026-10-21.md, base 30b75aaa).
- Q3 evidence: backend/docs/payment/stripe-sandbox-acceptance-20260920.md (browse START and JOIN through Hosted Checkout, local, 20 Sep); sprint-03 8 Oct log: Google sign on dev, Chrome DevTools shows blocbuy-token Secure and HttpOnly; hand mutation probes 12–13 Oct logs. No user acceptance testing with outside users; say so if asked.
- Q4 security: types only here (rubric 3.3). Tools and gates are on slide 28: SAST SonarCloud SpotBugs (CodeQL off), dependencies Trivy fs + Dependabot, image Trivy, IaC configuration Trivy config, secrets gitleaks.

Hand-over *"And this is the test that answers the performance target."*

Slide 25 Scripted performance test: tool, scenario, target

16:40:50–16:42:00 · 70 s

Speaker Liu Jin · covers guideline item **3a**

Say

Our scripted performance test is k6, pinned to one version and image digest. It runs against the SWE5001 ECS environment, which runs the same code; we never load the shared dev host, which also carries its database. The scenario steps the arrival rate from 10 to 200 requests a second, one minute per step, seven listing pages to three details, with no think time. Each step must keep p95 under 500 milliseconds, errors under 1 percent, and drop no scheduled requests. The proposal's target is 2 seconds at 100 concurrent users. By Little's law, 100 users who think for half a second send about 190 requests a second, so the top step covers the target with a gate four times stricter. Expiry within a minute was measured locally: 30 teams, worst lag 58 seconds. Yorik runs this in the system demo.

≈ 141 words · ≈ 65 s at 130 wpm · slot 70 s · if short of time: If over time, drop "pinned to one version and image digest".

Cues

- Tool and scenario: perf/k6/README.md §V2 and perf/k6/browse.js: grafana/k6:1.6.1 pinned by digest; ramping-arrival executor, 10, 20, 40, 60, 80, 100, 125, 150, 175, 200 iterations/s, 60 s each; one request per iteration, 7 list : 3 detail; no sleep; 100 preallocated VUs, 400 max; 10 s request timeout; gates per plateau p95 < 500 ms, error rate < 1%, zero dropped iterations (calibrate.mjs). Anonymous browse only: authenticated load needs real sessions.
- Environment: SWE5001 ECS, CloudFront d3lvq02o2e7tcn.cloudfront.net → ALB → ECS Fargate; the README says "Do not load-test shared SWE5006 dev". Say "SWE5001 environment, same code" every time; never call it a dev result.
- Target: proposal §6.3 "Interactive operations respond within 2 seconds under demonstration load (up to 100 concurrent users); team expiry is resolved within 1 minute of window lapse." Little's law N = X(R + Z): N = 100, Z = 0.5 s think, R ≈ 0.03 s gives X ≈ 189 requests/s. The 0.5 s think time is our assumption; with longer think times the offered load is lower.
- History, not the 5.3 result: SWE5001 V2 baseline 16 Oct (docs/swe5001/evidence/20261016-http-baseline-dashboard/README.md), one monolith task, two runs of 57,600 requests, zero errors, every plateau p95 ≤ 57 ms; saturation not observed, so 200 requests/s is a tested lower bound. Local Node workload 30 Sep (docs/perf/sprint3-baseline.md): 100 closed-loop VUs, 200 ms think, p95 browse 24 ms, JOIN intent 57 ms, My Orders 18 ms; 30 expiries, worst observation lag 58.18 s. Local only.
- Expiry on ECS: the V4 burst (2,000 teams, perf/k6/SCALING.md) has not been run; `[FILL: V4 expiry result on SWE5001 ECS, or say "local only"]`. The 5.3 run and its report: `[FILL: pre-run date, run label and summary path]`.

Hand-over *"Pan will now show where these tests gate the pipeline."*

Watch out

Never present a local or SWE5001 result as a SWE5006 dev result: say where each number ran.

Q&A (during the talk / 17:04–17:10)

Likely question	Answer: decision → reason → evidence or limitation (30–45 s)	Owner
How do you test payments	Stripe sits behind gateway ports (Checkout and Refund gateways). Unit tests swap the port; StripeRolloutAcceptanceIT (8 tests) runs the real HTTP endpoints, scheduler, Flyway schema and	Liu Jin → Yorik

without calling Stripe?	webhook signature checks, with only OAuth and the Stripe network calls substituted. Limitation: no live Sandbox refund on dev yet (#223, #517).	
Why Testcontainers rather than H2?	Real PostgreSQL 18, Redis 8.10 and ElasticMQ in containers. Our invariants live in the database: UNIQUE keys, ON CONFLICT DO NOTHING, guarded UPDATEs, role grants; H2 would accept or reject different SQL. Evidence: OrderRepositoryIT.duplicatePaymentIsRejectedAtTheDatabase. Cost: Docker is needed and the backend build takes about 13 minutes.	Liu Jin
Do you do TDD?	Not for everything, and we say so. Test-first for selected changes: the rate-limit checks failed (429 expected, 200 actual) before the code (3 Oct log); two red tests reproduced the join-quote defect before the fix (7 Oct log). Elsewhere tests come in the same pull request, and the 80% new-code gate makes them mandatory.	Liu Jin
What is your test coverage?	SonarCloud on dev, read 19 Oct: backend 99.4% overall and 99.2% on new code; frontend 96.1% overall and 96.2% on new code. The gate is at least 80% on new code (NFR-06). Limitation: coverage is not assertion quality, and we have no automated mutation testing. `[VERIFY: re-read SonarCloud on 25 Oct]`	Liu Jin
How do you test the frontend against the API?	Contract first. The backend regenerates contracts/openapi.json and fails on drift; openapi-diff fails a breaking change unless labelled. The frontend generates its client and its MSW handlers from that file (frontend ADRs 0003, 0011), and an unhandled request fails a test. Limitation: stubs prove the frontend half only.	Liu Jin → Lim
Why is the performance test not on dev?	Dev users exist only through Google sign-in, so 100 authenticated sessions cannot be scripted there, and dev is one host shared with its database. The scripted k6 test runs on the SWE5001 ECS environment, the same code (slide 25, run in §5); the Node workload ran locally on 30 Sep (docs/perf/sprint3-baseline.md). Neither is a dev result, and each is labelled.	Liu Jin
Is 200 requests a second the same as 100 concurrent users?	Not directly, so we convert. k6 drives an open arrival rate; the proposal counts users. Little's law N = X(R + Z): 100 users thinking 0.5 s with about 30 ms responses send about 190 requests/s, inside the 200 we drive, and our gate is p95 < 500 ms, four times stricter than 2 s. The 0.5 s think time is our assumption. The local closed-loop run (30 Sep) used exactly 100 users with 200 ms think: p95 at most 57 ms.	Liu Jin
Is the last-seat race test a unit test?	No, it is an integration test. StripeLocalIntegrationTest.twentyBuyersRaceForLastSeatAndNineteenRefundObligationsPersist races twenty paid buyers through the production intent and fulfilment ports against PostgreSQL 18 in Testcontainers. StripeCheckoutFlowTest.concurrentPaidBuyersCannotBothTakeLastSeat also runs on real PostgreSQL, but its provider and its group and order collaborators are test doubles (class javadoc), so we do not cite it as last-seat evidence. The pure unit tests of the same rule are PaidGroupDealTest (the aggregate's guards) and DealStatusTest (the transition table).	Liu Jin
Why no end-user acceptance testing?	Our quadrant three is manual checks by the team, logged: the Stripe Sandbox browser acceptance (20 Sep, local) and sign-in cookie checks on dev (8 Oct). We had no access to outside users within the course, and we do not claim UAT.	Liu Jin

4 CI/CD Pipeline Demo (10 minutes)

Clock	16:42:00 – 16:52:00 (slides 26–30 until 16:46:30, then the live pipeline demo on slide 31 until 16:52:00)
Owner	Panpakorn Siripanich (Pan); Lim approves as the frontend code owner
Slides	26, 27, 28, 29, 30, 31

Lecturer's presentation sequence · Presentation Guidelines · Briefing deliverables (V1.0)

4a — Describe the strategy to manage the source code and artifacts.
4b — Describe the DevSecOps pipeline and procedures.
4c — Highlight any technical findings and issues encountered.
4d — Design specification of the DevOps pipelines along with the use of appropriate tools with minimal security considerations.
4e — Backup: Record a video in advance in case of any technical issues during the live demo.

Tip

Slides 26–30 take 270 s. Pan opens the demo pull request at 16:41, during slide 25, so its checks (median 4.8 min) are green when the live part starts at 16:46:30. A backend change cannot finish its 16-minute PR checks inside the segment, so a backend PR is opened at ≈ 16:20 and merged before 16:40; the demo walks through both of its finished runs (rubric 4.1).

Slide 26 Source code and artefacts: one trunk, everything traceable to a commit 16:42:00–16:43:00 · 60 s

Speaker Pan · covers guideline item **4a**

Say

Thanks, Liu Jin. One monorepo holds code, contract, deployment and infrastructure. In the course's terms our branching is Branch by Feature: one short branch per story, a pull request, a squash merge onto the trunk, dev, so history stays linear. Instead of merging the trunk daily, a branch must be up to date to merge. A code owner approves and six checks must pass. People reach AWS through Identity Center with one-hour sessions; CI holds no AWS keys and assumes one OIDC role per job. The one exception is two IAM users for port forwarding. Artefacts trace to a commit: images tagged with the git SHA and deployed by digest, the contract committed, the client generated, frontend files content-hashed. We do not produce an SBOM yet.

≈ 126 words · ≈ 58 s at 130 wpm · slot 60 s · if short of time: If over time, drop "Instead of merging the trunk daily, a branch must be up to date before it can merge."

Cues

- Layout: git ls-files top level (22 Oct). Branching: EPAT 07 Continuous Integration v2.1 p41–44 (Develop on Trunk, Branch for Release, Branch by Feature, Branch by Team). Branch by Feature prerequisites we meet: one branch per story, reviewed merges, short life; the "merge trunk into every branch daily" prerequisite is replaced by the strict up-to-date rule. Linear history: no merge commits on dev since 12 Sep (git log --merges dev). SWE5001 adds a prod release branch; SWE5006 has none.
- Ruleset dev-require-review (gh api repos/PPsyrius/iss-groupbuy-team8/rulesets/22734681, read 22 Oct by the diagram check): 1 approval, require_code_owner_review, dismiss_stale_reviews_on_push, required_review_thread_resolution, strict, six required checks (build-quality, SonarCloud Backend, docker, gitleaks, frontend-check, SonarCloud Frontend), squash only. CODEOWNERS: /backend/ @l0sgAi (Liu Jin), /frontend/ and *.js @limjiechao, /.github/ @PPsyrius.
- People and CI: docs/deploy/aws-account-governance/README.md (Identity Center instance jiechao-aws, one-hour sessions from a permission set, SCP guardrails; IAM users blocbuy-dev-losgai with one active access key and blocbuy-dev-ppsyrius without keys, both BlocBuySsmPortForwarding). OIDC roles: ECR push (trust refs/heads/dev), backend deploy (environment:backend-dev), frontend deploy (environment:frontend-dev), AWS read-only 19 Oct.
  `[VERIFY: Identity Center MFA setting (the README does not state it)]`
- Artefacts: ci-backend.yml publish-ecr-dev tags git-${github.sha}-run-${run_attempt} and checks the pushed digest with aws ecr describe-images; .github/actions/deploy-ec2/action.yml rejects an image URI not pinned by sha256; deployment/backend/ecr-lifecycle-policy.json keeps 30; ECR immutable (operator guide). contracts/openapi.json + contract-check; frontend ADR 0003 (generated client, gitignored). ci-frontend.yml deploy-s3-dev: hashed assets max-age=31536000,immutable, index.html no-cache. Flyway V1–V28 in blocbuy-bootstrap. infra/terraform/README.md: state in S3 blocbuy-tfstate-402178232802 (encrypted) with DynamoDB lock; "CI never applies it"; the dev EC2 and ALB are outside Terraform. No syft/CycloneDX anywhere: no SBOM.

Hand-over *"Here is what runs on every pull request, and after the merge."*

Slide 27 Continuous integration: pull-request jobs and post-merge jobs

16:43:00–16:44:00 · 60 s

Speaker Pan · covers guideline item **4b**

Say

The pipeline in two lanes. **[click]** *On a pull request that changes the backend, build-quality runs every unit, architecture and integration test; contract-check regenerates the OpenAPI file and fails on drift or a breaking change; docker builds the image, boots it against PostgreSQL and Redis, and Trivy scans it.* **[click]** *The frontend runs its full check and Playwright on its own build; gitleaks runs on every pull request, Trivy on dependencies and configuration when those paths change. The six blue boxes are required.* **[click]** *After the squash merge, delivery rebuilds from the merged commit: package, push to ECR, deploy through SSM.* **[click]** *The frontend syncs to S3 and is smoke-tested through CloudFront. Our policy as code is Trivy's configuration rules plus our own deployment-bundle rules.*

≈ 121 words · ≈ 56 s at 130 wpm · slot 60 s · if short of time: If over time, drop the last sentence on policy as code (it is on slide 28).

Cues

- Diagram: src/diagrams/cicd-pr-vs-merge.svg (2320 × 1096; full view, then four 1:1 crops), checked against .github/workflows/ci-backend.yml, ci-frontend.yml, security.yml and .github/actions/deploy-ec2/action.yml on 22 Oct; element-by-element sources in the diagram spec (scratchpad, to be committed with the diagram notes).
- Triggers: pull_request → dev (ci-backend also on labeled/unlabeled, so the contract-change-approved label re-runs the gate); push to dev path-filtered (backend/**, deployment/backend/** with ENABLE_EC2_DEPLOY=true; frontend/**, contracts/** with ENABLE_S3_DEPLOY=true); security.yml weekly Monday 02:17 UTC. On a dev push security.yml runs only gitleaks. Jobs skip when their paths are untouched, and a skipped required job counts as passed.
- Contract drift: contract-check regenerates contracts/openapi.json from the verified JAR, fails on git diff, and runs openapi-diff --fail-on-incompatible unless the PR carries contract-change-approved; it is not a required check. Policy as code on SWE5006: trivy-config (Dockerfiles, Compose, Terraform, workflows, HIGH/CRITICAL) and deployment-bundle (ShellCheck; Compose may hold only the backend service with host networking; SSM bundle allowlist and size). Checkov and terraform test run only for infra/swe5001 (infra-swe5001.yml).
- Honest detail: package-dev rebuilds with -DskipTests and publish-ecr-dev builds the delivery image again without Trivy, so the deployed image is not the one the PR scanned; only the source tree is the same (strict up-to-date + squash). ECR scan on push per the operator guide. Open item 18.
- Timings (gh, 19 Oct, medians of successful runs): backend PR start → docker end 16.1 min; frontend PR → e2e-preview 4.8 min; push → backend deployed 3.1 min; push → frontend e2e-dev green 2.7 min. Table: B8.

Hand-over *"Security sits in each of those stages."*

Slide 28 Security controls by pipeline stage, and what each one catches

16:44:00–16:44:45 · 45 s

Speaker Pan · covers guideline item **4d**

Say

By stage. Before commit, hooks format, lint and type-check. At the pull request, gitleaks scans the pull request's commits, the full history weekly, and a code owner reviews. At build, SonarCloud and SpotBugs do static analysis, and Trivy and Dependabot watch dependencies. At test, Trivy scans the image and the configuration. At deploy: one OIDC role per job, an image pinned by digest, a read-only container. The gaps are on the right: CodeQL is off, there is no dynamic scan, and the image deployed after merge is rebuilt and not rescanned.

≈ 91 words · ≈ 42 s at 130 wpm · slot 45 s · if short of time: If over time, drop "Before commit, hooks format, lint and type-check."

Cues

- Stages: DSOEA 07 Secure CI-CD Pipeline v1.0 slide 3 (pre-commit, commit, build, test, deploy/production). Controls: lefthook.yml (oxfmt, oxlint, tsc on staged frontend files); security.yml gitleaks (required; on pull_request gitleaks-action v3 scopes the scan to the PR's commits, security.yml:164; the weekly schedule scans the full history from fetch-depth 0), workflow-lint (actionlint), trivy-deps (Maven + pnpm, HIGH/CRITICAL), trivy-config, deployment-bundle; ci-backend.yml build-quality (SpotBugs, Checkstyle, ArchUnit), SonarCloud Backend and Frontend (required, ≥ 80 % new-code coverage), docker job Trivy image scan (HIGH/CRITICAL, exit-code 1, required). .github/dependabot.yml: daily, cooldown, grouped updates. Actions pinned to 40-character SHAs (grep of .github, 19 Oct).
- Deploy and runtime: OIDC roles each scoped to one job (ECR push to one repository; SSM SendCommand of AWS-RunShellScript to one instance; one bucket and one distribution); deploy-ec2 rejects a non-digest image URI; deployment/backend/compose.yml read_only, cap_drop ALL, no-new-privileges; secrets in /etc/blocbuy/backend.env root 0600.
- Gaps: ENABLE_CODEQL=false and the code-scanning API answers 403 "not enabled" (docs/ci/ci-failure-audit-20261021.md); no DAST job anywhere; post-merge rebuild not rescanned (slide 27); four historical gitleaks findings open (PR #580); SSH 22 open on the dev host (AWS, 21 Oct; open item 9). Full list: B9.

Hand-over *"And this is where it all lands."*

Slide 29 Delivery: one environment, the code owner approves, the deploy verifies itself

16:44:45–16:45:45 · 60 s

Speaker Pan · covers guideline item **4b**

Say

SWE5006 has one environment, dev; the production stack was planned but never applied. The approver is the code owner on the pull request, and the merge is the release. Every merge deploys automatically to dev, which is not production. SWE5006 has no production, so there is no promotion gate; SWE5001's production waits for an approved promotion pull request, which the course calls Continuous Delivery. **[click]** *The browser talks only to CloudFront: files from a private bucket, API calls to a load balancer that refuses anything without our origin secret.* **[click]** *One EC2 host runs the container by digest, next to PostgreSQL and Redis on loopback: cheap and always on, but one point of failure.* **[click]** *A deploy runs through SSM, is health-checked on the host with automatic rollback, then checked publicly and smoke-tested.*

≈ 130 words · ≈ 60 s at 130 wpm · slot 60 s · if short of time: If over time, drop "files from a private bucket," and "then checked publicly and smoke-tested".

Cues

- Definitions: EPAT 07 Continuous Integration v2.1 p34 ("Deploy to Production is manually triggered" = Continuous Delivery) and p35 (pipeline with Approval). Continuous Deployment automates that production trigger. Dev is not production, so neither term applies to it: say "every merge deploys automatically to dev". SWE5006 has no production (#226), hence no promotion gate.
- Environments: GitHub environments backend-dev and frontend-dev allow only the dev branch and have no required reviewers, so the PR approval is the human gate. SWE5006 prod: infra/terraform/README.md (backend-app-stack plan of 21 creates, not applied, #48), #226 not planned. SWE5001: ruleset prod-release (promotion PR from dev, one CODEOWNERS approval, required promotion-source, service-images, infra-plan); ECS circuit breaker with rollback.
- Diagram: src/diagrams/dev-deployment.svg (UML deployment diagram; full view, then three 1:1 crops). AWS facts: CloudFront E3U5W7UWDL8LRV, S3 behind OAC, ALB default 403 with an X-Origin-Secret forward rule (read-only 19 Oct); EC2 i-0852b6c3475b19f05 t3.large, no ASG, SG 8589 only from the ALB group and 22 open to 0.0.0.0/0 (re-read 21 Oct). PostgreSQL 18 and Redis 8 on 127.0.0.1 (operator guide §1). Not drawn: the media CloudFront (open item 19).
- Verification: action.yml pre-deploy health → SSM send-command runs deployment/backend/bin/deploy.sh (pull by digest, compose up, on-host health-check.sh at 127.0.0.1:8589; on failure restarts the previous image and the job still fails, deploy.sh:101-141) → public health https://d1b3np88z657cp.cloudfront.net/api/actuator/health; frontend e2e-dev Playwright smoke through CloudFront. Limitation: no automatic rollback has fired yet and rollback-backend.yml has never run (gh, 19 Oct).
- Trade-off (SWE5001 feedback #589, database in its own subnet): on dev the database shares the app host; a deploy restarts the only API (502 on 3–4 Oct). Accepted for a low-cost always-on dev; the SWE5001 topology puts RDS in private subnets (backup B10).

Hand-over *"Here is what building this taught us."*

Slide 30 Technical findings: three issues, their root causes and fixes

16:45:45–16:46:30 · 45 s

Speaker Pan · covers guideline item **4c**

Say

Three findings. First, our hand-built dev environment drifted from its design: an outdated OIDC subject format and a placeholder instance ID in the deploy policy, so the first nine deploys failed. We fixed each and wrote the production stack in Terraform, where a plan shows every value. Second, Dependabot upgraded the SpotBugs engine without its plugin, and the build stayed red for two days; now both move together and a test guards the POM. Third, the weekly secret scan failed on twenty file hashes; we fingerprinted exactly those, and four findings stay open.

≈ 93 words · ≈ 43 s at 130 wpm · slot 45 s · if short of time: If over time, drop the third finding from speech; it stays on the slide.

Cues

- (1) docs/deploy/prod/backend-prod-terraform-deployment.md lines 15–23 (dev incident table: OIDC sub format, placeholder instance i-0123456789abcdef0 in the SSM policy, aws CLI missing on the host, 8589 open to 0.0.0.0/0, hand-edited variables); fix commit 121a0733 (immutable subject format). Deploy history (gh, 19 Oct): 9 failures 14–17 Sep ("Instances not in a valid state"), 88 of 97 green, none failed since. Port 8589 now only from the ALB group (AWS, 21 Oct).
- (2) docs/ci/ci-failure-audit-20261021.md: run 39463373272 (20 Oct) missing BCEL PermittedSubclasses with plugin 4.10.4.1 and engine 4.10.5, repeated 19–20 Oct; fixed in PR #580 (commit 0baf2052): backend/pom.xml plugin 4.10.5.0 + engine 4.10.5, Dependabot group spotbugs, .github/tests/backend_quality_test.py.
- (3) Same audit: weekly security.yml gitleaks failed on 21 and 28 Sep, 5, 12 and 19 Oct; 24 generic-api-key findings, 20 of them SHA-256 file digests in scripts/stripe-dev-contract-baseline.json, each re-hashed and matched, then listed by exact fingerprint in .gitleaksignore; 4 Sonar badge-token findings in backend/README.md remain, so the full-history scan still fails.
- Also true, if asked: the Stripe webhook on dev never confirms a payment (#517, open); CodeQL ingestion fails because code scanning is unavailable on this private repository (run 34938330654, 15 Sep); an Actions budget stopped a run on 20 Oct (run 39442654274).

Hand-over *"Now let me show you one change going through."*

Slide 31 Live: one change from pull request to dev 16:46:30–16:52:00 · 330 s

Speaker Pan · covers guideline item **4e**

Say *Now live: the pull request I opened before this segment. Its checks ran during the last slides, so let's approve it, merge it and watch it reach dev. The backend run takes sixteen minutes, so it was triggered beforehand and we walk through it finished.*

≈ 45 words · ≈ 21 s at 130 wpm · slot 330 s

Cues

- Opens the live pipeline demo: steps P1–P6 in the script, §4 (16:46:30–16:52:00). Pan opens the demo PR at 16:41, during slide 25, so its checks (median 4.8 min) are green by 16:46:30.
  `[CONFIRM: the demo change, its author and the approving code owner (frontend → Lim)]`
- Rubric 4.1: a full backend run exceeds 8 min (PR checks median 16.1 min), so it is triggered beforehand and walked through finished:
  `[FILL: backend PR run and its post-merge deploy run of 26 Oct]`.
- Rubric 4.6:
  `[FILL: backup video link, recorded on dev by 24 Oct, in the cloud and on a USB drive]`

Hand-over (demo) "Here is the pull request."

Live pipeline demo (16:46:30 – 16:52:00, 330 s)

Step	Shows	Rubric
P1	The pull request and its required checks	Rubric **4.1**, **4.2** (branch, PR, Conventional Commit, code-owner rule), **4.3** (PR-time jobs)
P2	Code-owner approval	Rubric **4.2**, **4.4** (who approves)
P3	Squash merge	Rubric **4.2** (linear history), **4.4** (the merge is the release)
P4	Delivery runs; the backend PR and deploy, triggered beforehand	Rubric **4.1** (pre-triggered long run), **4.3** (post-merge jobs), **4.4** (digest, SSM, health check)
P5	Verified: the change is live on dev	Rubric **4.4** (how a deployment is verified)
P6	Wrap up and hand over	Rubric **4.1–4.4** summary

Preconditions (checked at 15:30 by Pan)

- Demo branch pushed and up to date with `origin/dev`; PR body ready; no PR opened yet. The backend merge (by 16:40) moves `dev`, so at 16:40–16:41 Pan rebases the demo branch onto the new `origin/dev`, pushes it and only then opens the PR; its checks start at 16:41 on the updated branch.
  `[CONFIRM: the copy change, its page, its author and the approving code owner (open item 14)]`
- Backend pre-run: a small backend PR opened at ≈ 16:20 by its author, approved by Liu Jin (backend code owner) and squash-merged by 16:40 so its deploy finishes before the segment.
  `[CONFIRM: the backend change and its author]`
- Merge freeze in the team chat: nobody merges into `dev` from 15:00 to 17:15 except these two PRs. A moved `dev` makes the branch out of date and re-runs the checks (strict required checks), which is why the demo branch is updated after the backend merge and before its PR opens, and why nothing else may merge after 16:40.
- Latest `ci-frontend` and `ci-backend` runs on `dev` green (A.1); deploy switches `ENABLE_S3_DEPLOY` and `ENABLE_EC2_DEPLOY` unchanged; GitHub Actions budget not exhausted (a budget stop hit run 39442654274 on 20 Oct).
- Lim signed in to GitHub on his own machine with the PR list open; he is not sharing at 16:47.
- Backup video (rubric G6, 4.6): recorded on dev by Sat 24 Oct, 1080p, one clip per step P1–P6, stored in the cloud and on a USB drive, open in a player on Pan's machine and tested with Zoom "share sound" off.

`[FILL: recording link and USB holder]`

Screen layout (Pan's browser window, shared from 16:46:30; browser zoom 125–150 %, rubric V7)

Tab	Content	Prepared state
1	GitHub: the demo pull request	Opened at 16:41; Files changed tab ready
2	GitHub Actions: `ci-frontend` filtered to branch `dev`	Refresh after the merge to see the new run
3	GitHub Actions: the backend PR run (pre-triggered)	Run summary open; build-quality one click away
4	GitHub Actions: the backend push run on `dev` (deploy)	Run graph open; deploy job one click away
5	Dev site `https://d1b3np88z657cp.cloudfront.net`	On the page the change touches; DevTools closed
Player	Clips P1–P6 (24 Oct)	Paused at the start of clip P1

Step by step

Step P1 The pull request and its required checks — 16:46:30 – 16:47:30

Proves Rubric **4.1**, **4.2** (branch, PR, Conventional Commit, code-owner rule), **4.3** (PR-time jobs) · Pan

Operate
1. Tab 1: the demo PR opened at 16:41 → **Files changed**: the one-line frontend change.
2. Switch to **Checks**: the required frontend-check, SonarCloud Frontend and gitleaks green; e2e-preview (not required) green; the backend jobs pass after skipping (no backend path changed).
3. Point at the merge box: review required from a code owner.

Say *This is the pull request I opened at 16:41: one line in the frontend, on a short branch with a Conventional Commit title. These are the required checks from slide 27: the frontend check with every unit test, SonarCloud and the secret scan. Playwright also runs against this build, but it is not a required check. The backend jobs see no backend change and pass straight through. The merge button stays locked until a code owner approves.*

Expect Checks green or finishing (median 4.8 min from 16:41, so ≈ 16:46); merge box asks for a code-owner review.

Fallback Checks still running: talk through them and move on at 16:47:30 regardless. A check red: say what it caught in one sentence, then `CLIP P2`; leave the PR open and never merge it on air.

Step P2 Code-owner approval — 16:47:30 – 16:48:00

Proves Rubric **4.2**, **4.4** (who approves) · Pan; Lim approves

Operate
1. Pan posts `APPROVE` in the backchannel. Lim (not sharing) approves from his own GitHub session: **Review changes → Approve**, or `gh pr review <n> --approve`.
2. Tab 1: refresh; the approval from @limjiechao appears; the merge box turns green.

Say *Frontend files belong to Lim in CODEOWNERS, so his approval is the one that counts; mine would not. If anyone pushes again, the approval is dismissed and the checks run again.*

Expect One approving review by a code owner; all checks passed; squash merge enabled.

Fallback Lim cannot approve within 30 s: nobody else is a frontend code owner, so do not ask anyone else. `CLIP P2`.

Step P3 Squash merge

16:48:00 – 16:48:20

Proves Rubric **4.2** (linear history), **4.4** (the merge is the release) · Pan

Operate
1. Tab 1: **Squash and merge**, keep the Conventional Commit title, confirm. Only this PR.
2. Say the merge time aloud.

Say *Squash merge: one commit on dev with the pull request number, so history stays linear. That push is the release; nothing manual follows.*

Expect PR merged; leave the branch-delete button alone.

Fallback "Base branch was modified" (dev moved): do not update the branch on air, it would re-run five minutes of checks. `CLIP P3`; merge after 17:15.

Step P4 Delivery runs; the backend PR and deploy, triggered beforehand

16:48:20 – 16:49:50

Proves Rubric **4.1** (pre-triggered long run), **4.3** (post-merge jobs), **4.4** (digest, SSM, health check) · Pan

Operate
1. Tab 2: Actions → `ci-frontend` → the new run on `dev`: package-dev → deploy-s3-dev → e2e-dev.
2. Tab 3: the backend PR run triggered at ≈ 16:20 `[FILL: run id]`: build-quality (test count in the summary), contract-check, docker boot and Trivy, SonarCloud gate.
3. Tab 4: its post-merge deploy run `[FILL: run id]`: publish-ecr-dev (tag git-<sha>-run-<n>, digest checked), deploy-existing-ec2 (SSM, on-host health check, public health).

Say *Delivery rebuilds from the merged commit: the frontend goes to S3, index.html is invalidated, then Playwright smoke-tests the real site. While it runs, here is a backend change that went through earlier: sixteen minutes of checks with every test, the contract check and the image scan; after its merge the image went to ECR, was deployed by digest through SSM to the one host, and was health-checked, with automatic rollback if the check had failed.*

Expect Frontend run in progress (median 2.7 min from push to e2e-dev green); both backend runs green.

Fallback Frontend run not started within 60 s: refresh once; then `CLIP P4`. Backend runs missing: open the B8 timings and say the medians.

Step P5 Verified: the change is live on dev

16:49:50 – 16:51:20

Proves Rubric **4.4** (how a deployment is verified) · Pan

Operate
1. Tab 2: e2e-dev green.
2. Tab 5: `https://d1b3np88z657cp.cloudfront.net` on the changed page → reload → the new text. `[CONFIRM: the page and the copy change (open item 14)]`
3. Read the elapsed time off the run: merge → e2e-dev green.

Say *Smoke test green, and here is the change on the dev site, about three minutes after the merge. That is the verification: the deploy job checks health, and Playwright checks the real site through CloudFront.*

Expect New text visible after a reload (index.html is served no-cache and invalidated); run green.

Fallback e2e-dev red: show the failed step honestly, it is the gate doing its job, then `CLIP P5`. Text not visible: one more reload, then `CLIP P5`.

Step P6 Wrap up and hand over 16:51:20 – 16:52:00

Proves Rubric **4.1–4.4** summary · Pan

Operate
1. Stay on tab 5 (the dev site) while speaking.
2. At 16:52:00: stop sharing; Yorik starts sharing (SHARE).

Say *So one change went from pull request to dev in about ten minutes, past six required checks, a code-owner review and an automatic smoke test, with no AWS keys stored in GitHub. Yorik will now show what that system does for two buyers.*

Expect Hand-over on time.

Fallback If behind, say only the last sentence.

Never live

Clicking Merge on anything other than the demo PR; “Update branch” on the demo PR; re-running a failed job; running `rollback-backend.yml` or any `workflow_dispatch`; opening Settings → Secrets and variables or the ruleset editor; pushing to `dev`; any AWS console or SSM session on the shared screen. If the PR is not merged by 16:49, leave it open and merge after 17:15, so a deploy never lands during the system demo.

Tip

Narrate results, not clicks: “checks green in four minutes fifty”, not “now I click the checks tab”. No troubleshooting on air.

Q&A (during the talk / 17:04–17:10)

Likely question	Answer: decision → reason → evidence or limitation (30–45 s)	Owner
Why EC2 and not ECS for dev?	Dev reuses one existing EC2 host with the monolith in Docker Compose (operator guide; backend ADR 0008): SWE5006 needs one complete, always-on system at low cost, and the monolith fits one t3.large. Limitation: one host, so a deploy restarts the API (502 on 3–4 Oct), and PostgreSQL and Redis share the host outside the pipeline.	Pan
How do you roll back?	Automatic: deploy.sh health-checks the new container and restarts the previous digest if it fails; the workflow also checks public health. Manual: rollback-backend.yml (reason, confirmation, same concurrency group). Limitation: rollback-backend.yml has never run and no automatic rollback has fired. Flyway is never reversed, so migrations stay backward-compatible.	Pan
How are secrets handled?	CI holds no AWS keys: OIDC gives short-lived roles per job. The repository has three Actions secrets, none for AWS or the app (SONAR_TOKEN, TYPESAFE_API_KEY, BOARD_SNAPSHOT_TOKEN; gh secret list, 19 Oct). Dev application secrets live in /etc/blocbuy/backend.env (root, 0600). Limitation: the account still has two IAM users for SSM port forwarding, one with an active key (governance README).	Pan
Why is CodeQL off?	The job exists in security.yml behind ENABLE_CODEQL, which is false: code scanning is not enabled for this private repository (GitHub API, 19 Oct). SAST is covered by SonarCloud on every PR plus SpotBugs in the build. We list it as a gap (B9).	Pan
How long from merge to live?	Medians from gh on 19 Oct: backend about 3.1 minutes from the push run starting to the deploy job ending (88 runs); frontend about 2.7 minutes including the e2e-dev smoke (94 runs). PR checks take about 16 minutes for backend and 5 for frontend. Per-workflow table: B8.	Pan
How do you manage dependencies?	Dependabot checks Maven, npm, Docker and GitHub Actions daily with a cooldown and grouped updates (SpotBugs engine and plugin move together since PR #580); majors of Spring, Java, React and TypeScript are ignored on purpose. Every update passes the same required checks. Trivy scans dependencies on change and weekly. Actions are pinned to commit SHAs.	Pan → Lim

Is the deployed image the one you tested?	Not byte for byte. The squash merge and the strict up-to-date rule make the merged tree the tested tree, but package-dev rebuilds it with -DskipTests and publish-ecr-dev builds the delivery image again without a Trivy scan; ECR scans it on push. That falls short of "build once, promote"; frontend ADR 0004 still describes promotion (open item 18).	Pan
Is this Continuous Delivery or Continuous Deployment?	Both terms are about production (EPAT 07 CI p34–35: "Deploy to Production is manually triggered"; automated, it is Continuous Deployment). SWE5006 has no production (#226), so there is no promotion gate: every merge deploys automatically to dev, a non-production environment, after the code owner approves the PR. SWE5001 production is Continuous Delivery: a promotion PR into prod needs a CODEOWNERS approval and three required checks.	Pan
Why no DAST or SBOM?	Not built: no dynamic scan runs against the deployed site, and no SBOM is generated. Dependency risk is covered by Trivy on the lockfiles and Dependabot; both gaps are listed in B9.	Pan

5 System Demo (12 minutes, then 6 minutes of buffer)

Clock	16:52:00 – 17:04:00 (slide 32 to 16:52:30; live demo on slide 33 to 17:01:00; slides 34–36 to 17:04:00) · 17:04–17:10 buffer for parked questions
Owner	Yu Qianshuo (Yorik); Wang Ziang (Eric) drives the joiner's browser; Lim chairs the buffer
Slides	32, 33, 34, 35, 36, 37

Lecturer's presentation sequence · Presentation Guidelines · Briefing deliverables (V1.0)

5a — Demonstrate some of the significant use cases of the solution.
5b — Demonstrate how the solution deals with some of the important quality attributes e.g. scalability (such as demonstrating auto-scaling), performance (show using some scripted performance testing), maintainability (show/run the DevOps pipeline).
5c — Backup: Record a video in advance in case of any technical issues during the live demo.

Tip

Eric shares one screen at 16:52:00 (SHARE from Pan) with the deck in the Leader window, tab 1, and operates every window; Yorik speaks throughout. The expiry bloc is started at 16:05 so that D4 shows a refund that no one triggered by hand.

Slide 32 System demo: two buyers, one bloc, and the qualities it shows

16:52:00–16:52:30 · 30 s

Speaker Yorik · covers guideline item **5a**

Say

Thanks, Pan. I'm Yorik. Two buyers on our dev site: a leader starts a bloc and shares it, a friend joins, the team succeeds. A second bloc, started before the talk, expires and refunds itself. Then three checks you can't see in the flow: security, the last-seat race and performance. Every step says where it runs.

≈ 56 words · ≈ 26 s at 130 wpm · slot 30 s · if short of time: If over time, drop "Every step says where it runs." (it is the slide's right column).

Cues

- 16:52:00. Eric is sharing one screen with two Chrome profiles, "Leader" (account A) and "Joiner" (account B), and operates both; Yorik narrates (two users: SWE5001 feedback #590). Deck in the Leader window, tab 1, on slide 32.
- Storyline from #464: invite → START/JOIN with Stripe Sandbox → /account orders and blocs → expiry refund. Use cases: UC1 browse, UC2 Google sign-in, UC3 start, UC7 invite, UC4 join (team of two succeeds, UC5), UC5 expiry, UC6 refund, UC8 My blocs. Quality attributes are slide 3's: correctness under concurrency, refund idempotency, expiry ≤ 1 min, security, performance (slide 34), maintainability (§4, slide 36).
- Environment column on the slide follows the 23 Oct go/no-go (open item 1): `[CONFIRM: GO or NO-GO for steps D1, D2 and D4; with NO-GO those rows read "recorded" or "local" and the say changes to "the payment steps are recorded"]` .

Hand-over *"Our leader is already signed in with Google."*

Decision: live or recorded payment steps (go/no-go, Fri 23 Oct, 18:00)

Condition (Yorik decides with Lim; posted in #464)	GO: live on dev	NO-GO: recorded or local, said as such

#517 fixed: a new START on dev reaches paid within one minute and shows under My blocs	D1, D2 live; fallback = the 24 Oct clips	D1, D2 from the 24 Oct clips if they exist, else from local evidence (B11), said as "verified locally, not yet on dev"
#223 done: Checkout and scheduling on for dev (`BLOCBUY_SCHEDULING_ENABLED`; default false in application-ec2.yml); a `US34-04` bloc started on 24 Oct expired within one sweep and refunded	D4 live with the 16:05 bloc	D4 from B11 (local); slide 32 rows read "recorded" or "local"
Full dry run on dev on Sat 24 Oct passes in order, timed, and is recorded	Rehearse at the real clock on 25 Oct	Only the NO-GO path is rehearsed
D3, D5, D6 do not depend on payments	live	live (D3 on an account that already has orders)

Watch out

No payment has been confirmed and no refund proven on dev as of 22 Oct (#517, #223 open). Every dev payment step here is conditional on the 24 Oct dry run.

`[CONFIRM: go/no-go result of 23 Oct, per step D1, D2, D4 (open item 1)]`

Preconditions (checked at 15:30 by Eric and Yorik)

- Health `UP` (A.1); latest runs on `dev` green; merge freeze in force; the §4 demo PR is the only change that may land.
- Two Google buyer accounts signed in on Eric's machine in two Chrome profiles, "Leader" (account A) and "Joiner" (account B); the same two on Yorik's machine as standby.
  `[CONFIRM: which two accounts and the display names shown on screen (open item 15)]`
- At **≈ 16:05** (latest 16:07, so the refund lands before the 16:50 check and §5 starts at 16:52) Yorik, as account B, starts a bloc on `US34-04` and pays; nobody joins; receipt bookmarked in the Joiner window, tab 3. It is due ≈ 16:45; the sweep (every minute) expires it and the refund worker (every 30 s) refunds it ≈ 16:46–16:48. Post `EXPIRY OK` at 16:10 and `REFUND OK` / `REFUND LATE` at 16:50.
- Listings converged: `scripts/seed-dev-demo-listings.sh --check` passes for the four `US34-*` listings (≤ 12:00, Yorik).
- Terminal with font ≥ 18 pt and the curl line ready (D5); Joiner DevTools closed until D5.
- IDE on a local checkout of `dev`, font ≥ 18 pt, with the D6 test run green after 15:00 and its file open; Docker running (rubric G10).
- Backup clips D1–D6 recorded on dev on Sat 24 Oct at 1080p, stored in the cloud and on a USB drive, open in a player on Eric's machine (copy on Yorik's), tested with Zoom "share sound" off (rubric G6, 5.5).
  `[FILL: clip folder link and USB holder]`
- Browser zoom 125–150 % (rubric V7); Do Not Disturb on; no password manager, email or bookmarks bar visible.

Screen layout (Eric's shared screen)

Window · tab	Content	Prepared state
Leader · 1	Deck `swe5006-presentation.html#32`	Slide 32; slides 34–37 follow
Leader · 2	Listing `US34-04` on `https://d1b3np88z657cp.cloudfront.net`	Signed in as account A
Joiner · 1	Empty tab for the invite link	Signed in as account B
Joiner · 2	`/account` (account B)	Loaded

Joiner · 3	Receipt of the 16:05 payment	Bookmarked at 16:10; reload before D4
Terminal	curl for D5	Command typed, not run
IDE	D6 test, green	Test file and PaidGroupDealMapper.xml open
Player	Clips D1–D6 (24 Oct)	Paused at clip D1

Live system demo (16:52:30 – 17:01:00, 510 s)

Step	Shows	Rubric
D1	Leader starts a bloc and shares the invite	Rubric **5.1** (UC2, UC1, UC3, UC7), **5.2** security
D2	A friend joins through the invite: the team succeeds	Rubric **5.1** (UC4, UC5, UC7), **5.2** correctness under concurrency
D3	My blocs and my orders	Rubric **5.1** (UC8)
D4	The 16:05 bloc expired and refunded itself	Rubric **5.1** (UC5, UC6), **5.2** unattended expiry and refund, recorded once (≤ 1 min is measured locally, slide 34)
D5	Security checks on the deployed API	Rubric **5.2** security (slide 3)
D6	Twenty buyers, one last seat: the test	Rubric **5.2** correctness under concurrency; rubric G10 (IDE on hand)

Slide 33 Live: two buyers, one bloc

16:52:30–17:01:00 · 510 s

Speaker Yorik · covers guideline item **5c**

Say

Steps D1 to D6 follow the run book. We narrate what changes, not what we click, and any step that stalls for twenty seconds switches to its recording.

≈ 28 words · ≈ 13 s at 130 wpm · slot 510 s

Cues

- Opens the live system demo: steps D1–D6 in the script, §5 (16:52:30–17:01:00). Eric shares and operates both profiles; Yorik speaks. Go/no-go table and NO-GO path in the script.
- D1 start (UC2, UC1, UC3, UC7) 120 s · D2 join, team succeeds (UC4, UC5, UC7) 120 s · D3 My blocs and orders (UC8) 75 s · D4 the 16:05 bloc expired and refunded (UC5, UC6; unattended expiry and one refund record; the ≤ 1 min bound is measured locally, slide 34) 90 s · D5 security checks (401, 403, 400, HttpOnly) 60 s · D6 last-seat race, local test in the IDE (correctness under concurrency) 45 s.
- Rubric 5.5: `[FILL: backup video link; clips D1–D6 recorded on dev by Sat 24 Oct, stored in the cloud and on a USB drive]` .

Hand-over (demo) "Here is the listing our leader starts from."

Step by step

Step D1 Leader starts a bloc and shares the invite — 16:52:30 – 16:54:30

Proves Rubric **5.1** (UC2, UC1, UC3, UC7), **5.2** security · Eric operates, Yorik speaks

Operate

1. Leader window (account A, signed in with Google), tab 2: listing `US34-04` “Universal Travel Adapter” (team of 2, 40 min; 29.00, team price 22.00).
2. **Start a bloc** → the quote sheet shows the team price → **Continue to payment**.
3. Stripe Checkout with its Sandbox banner: test card 4242 4242 4242 4242, any future expiry, any CVC → Pay.
4. Back on BlocBuy: the receipt turns from “Waiting for payment confirmation” to paid; open the bloc: 1 of 2, countdown running.
5. Copy the invite link (`/products/{id}?deal={dealId}&ref={ref}`).

Say *Our leader is signed in with Google. Every listing has two prices; she starts a bloc at the team price and pays first, on Stripe's own page, in Sandbox mode: a test card, no real money. Notice the receipt waits: BlocBuy believes a payment only when Stripe's signed webhook arrives, not when the browser comes back. Now the bloc exists, one of two, and she copies her invite link.*

Expect Receipt paid within one minute; bloc open at 1 of 2; invite link copied.

Fallback Checkout does not open within 20 s, or the receipt still waits after 60 s (the #517 symptom): Yorik posts `CLIP D1` and says “*That is the open issue on slide 4; here is the same step recorded on dev on 24 Oct.*” No dev recording: Plan C.

Step D2 A friend joins through the invite: the team succeeds — 16:54:30 – 16:56:30

Proves Rubric **5.1** (UC4, UC5, UC7), **5.2** correctness under concurrency · Eric operates, Yorik speaks

Operate

1. Joiner window (account B), tab 1: paste the invite link.
2. Point at the banner that names the sharer (account A’s display name).
3. **Join** → quote → Stripe Checkout → 4242 card → Pay → receipt paid.
4. Product page or the receipt’s bloc link: 2 of 2, **succeeded**; both orders confirmed.

Say *A friend opens the link. The banner names who shared it, which is how the join is credited. She joins and pays the same way. The join rules run twice, at the quote and again after payment, inside the team, under a row lock: if the last seat had gone in between, she would get a refund obligation, never a seat that doesn't exist. Two of two: the team succeeded, and both orders are confirmed in the same transaction.*

Expect Sharer banner; receipt paid; bloc 2 of 2, succeeded.

Fallback `CLIP D2` with the same sentence as D1. No recording: Plan C.

Step D3 My blocs and my orders

16:56:30 – 16:57:45

Proves Rubric **5.1** (UC8) · Eric

Operate

1. Joiner window, tab 2: `/account`. My blocs → **Completed**: the bloc just filled.
2. My orders below: its confirmed order.
3. Click **Refunded**: the 16:05 bloc (D4); point at the URL (`?blocs=…`); reload: the chip survives.

Say *Her account page: My blocs on top, with Active, Completed and Refunded, and her orders below. The chip lives in the URL, so a reload, the Back button or a shared link keeps the view. Under Refunded, a bloc she started before the talk.*

Expect Completed shows the new bloc; My orders lists the confirmed order; the chip survives a reload.

Fallback Page empty: reload once; then `CLIP D3`. With a NO-GO this step runs live on an account that already has orders.

Step D4 The 16:05 bloc expired and refunded itself

16:57:45 – 16:59:15

Proves Rubric **5.1** (UC5, UC6), **5.2** unattended expiry and refund, recorded once (≤ 1 min is measured locally, slide 34) · Eric operates, Yorik speaks

Operate

1. Joiner window, tab 2, Refunded: the `US34-04` bloc started at ≈ 16:05, chip “Closed”; its order cancelled.
2. Tab 3: its receipt `/payments/{paymentId}` (bookmarked at 16:10), reloaded: the refund status.
3. Point at the deadline (≈ 16:45).
   `[VERIFY: whether the bloc row or receipt shows the deadline and the closing time]`
4. Never open the Stripe dashboard on the shared screen.

Say *At five past four she started this bloc, a team of two with a forty-minute window, and nobody joined. At its deadline the expiry sweep, which runs every minute, closed it; in the same transaction the order was cancelled and exactly one refund obligation was written. A worker sent that refund to Stripe with an idempotency key built from the obligation, so a retry after a crash returns the same refund instead of a second one. We never touched the deadline; the timer did all of it.*

Expect Bloc under Refunded (“Closed”); order cancelled; the receipt shows the refund.

Fallback `REFUND LATE` at 16:50 and still none now: Yorik says “the worker retries every 30 seconds; it may still be in flight”, then `CLIP D4`. No dev recording: open **B11** and say “*verified locally with a real Sandbox refund on 20 September, not yet on dev.*”

Step D5 Security checks on the deployed API

16:59:15 – 17:00:15

Proves Rubric **5.2** security (slide 3) · Eric operates, Yorik speaks

Operate

1. Terminal (18 pt): `curl -s https://d1b3np88z657cp.cloudfront.net/api/v1/orders/me` → `401`, `COMMON_401_001` (SaTokenInterceptor, SaTokenConfig).
2. Joiner window, new tab: `https://d1b3np88z657cp.cloudfront.net/api/v1/admin/products` → `403`, `COMMON_403_001` (a buyer fails `StpUtil.checkRole(ADMIN)`, AdminListingController).
3. Joiner DevTools console: `document.cookie` → no `blocbuy-token` (HttpOnly). Never open the Application tab: it shows cookie values.
4. Console: `fetch('/api/v1/payments/intents', {method: 'POST', headers: {'Content-Type': 'application/json'}, body: '{}'}).then(r ⇒ r.status)` → `400` (no `X-BlocBuy-Request` marker; GlobalExceptionHandler.handleMissingHeader).

Say *Three checks you can't see in the flow. No session: 401. A signed-in buyer on an admin endpoint: 403, because admin calls check the role and the permission. The session cookie is HttpOnly, so page scripts can't read it. And a money write without our custom header is refused: a cross-site form can't add that header.*

Expect 401, 403, no token in document.cookie, 400.

Fallback Any check differs: say what came back, one sentence, and move on; the Q&A row "Is 401 vs 403 vs 400 deliberate?" has the mapping.

Step D6 Twenty buyers, one last seat: the test

17:00:15 – 17:01:00

Proves Rubric **5.2** correctness under concurrency; rubric G10 (IDE on hand) · Eric shows, Yorik speaks

Operate

1. IDE (18 pt) on a local checkout of `dev`: `StripeLocalIntegrationTest.twentyBuyersRaceForLastSeatAndNineteenRefundObligationsPersi` already run green before the talk (Docker, PostgreSQL 18 in Testcontainers).
2. Show the assertions: 2 members, 2 orders, 19 refund obligations, `current_size` 2, stock 98.
3. One click to `PaidGroupDealMapper.xml`: `SELECT … FOR UPDATE` and the guarded `UPDATE … WHERE current_size = …`.

Say *The last seat we don't race on shared dev. This test does it on every pull request that changes the backend: twenty paid buyers race for the last seat of a team of two, through the production code against a real PostgreSQL. One wins; nineteen get a refund obligation, and capacity is never exceeded. The row lock and the guarded update are why. This is a local test, not a dev result.*

Expect Green test with its assertions visible; then back to the deck, slide 34.

Fallback IDE not ready: show the test on GitHub (read-only) and its last green CI run.

Never live

Editing any deadline, price or listing on shared dev; any SSM command, database client or AWS console on the shared screen; the Stripe dashboard (keys and webhook secrets live there); the DevTools Application or Network tab with cookies or headers visible; racing two buyers or replaying a webhook on dev; a second payment to "try again"; pointing k6 at dev.

Plan C

A step fails live and has no recording: show its local evidence (B11 for payment and refund, the D6 test for the race, B12 for load) and say: *"verified locally, not yet on dev."* Never present a local result as a dev one.

After the demo (17:01:00 – 17:04:00)

k6 pre-run for slide 34 (rubric 5.3)

- **Sat 24 Oct**, Pan: bring up the SWE5001 environment (applied per session, torn down nightly; open item 16), run `perf/k6/browse.js` (k6 1.6.1 by digest; ten 60 s plateaus, 10 → 200 requests/s) against the SWE5001 CloudFront, keep the k6 summary and the dashboard export, write the run into the day log, tear the environment down. `[FILL: pre-run date, label and summary path]`
- Slide 34's table is filled from that summary before the 25 Oct rehearsal; on the day the report file is opened from Eric's machine if asked. Never point k6 at the SWE5006 dev host.

Slide 34 Performance against the target: the scripted k6 run 17:01:00–17:03:00 · 120 s

Speaker Yorik · covers guideline item **5b**

Say *Performance, with the scripted test from slide 25. A full run is ten one-minute steps, so we ran it before the talk and show its report; this is the SWE5001 ECS environment running the same code, not the dev host. Each row is one step of the arrival rate, from ten to two hundred requests a second, with its p95 latency, its error rate and dropped requests. The proposal's target is two seconds at a hundred concurrent users. By slide 25's conversion, a hundred users thinking half a second offer about a hundred and ninety requests a second, so the rows at and above that rate are the target load. Our gate is stricter: p95 under half a second on every step. Expiry within a minute comes from the one-minute sweep and the local measurement of 58 seconds. What we don't claim: authenticated paths such as join under load on a deployed environment, and anything about the dev host.*

≈ 159 words · ≈ 73 s at 130 wpm · slot 120 s · if short of time: If over time, drop the last sentence ("What we don't claim..."); it is in the cues and Q&A.

Cues

- Pre-run (rubric 5.3: a live run over ≈ 2 min is pre-run): k6 1.6.1 perf/k6/browse.js against the SWE5001 CloudFront d3lvq02o2e7tcn.cloudfront.net, ten 60 s plateaus 10 → 200 iterations/s (≈ 10 min plus setup). Run by Pan on Sat 24 Oct with the summary and dashboard export kept; re-run on Mon 26 Oct ≤ 14:00 only if the environment is up and the first run failed. `[FILL: run date, label and summary path of the 24 Oct pre-run]`
- Results: `[FILL: p95 per plateau (ms), error rate, dropped iterations, VUs used at 200/s; pass or fail against p95 < 500 ms, errors < 1 %, zero dropped]`. Show the k6 summary or the dashboard tab, never a hand-typed number.
- Environment prerequisite: the SWE5001 environment is applied per session and torn down nightly (research-aws.md; infra/swe5001 env-up.sh). Pan brings it up on 24 Oct for the pre-run (open item 16); it does not need to be up during the talk, because slide 34 shows the saved report. Never point k6 at the SWE5006 dev host (perf/k6/README.md: "Do not load-test shared SWE5006 dev").
- Mapping (slide 25): Little's law N = X(R + Z), N = 100, Z = 0.5 s (our assumption), R ≈ 0.03 s → X ≈ 189 requests/s. History, labelled: V2 baseline 16 Oct on ECS, every plateau p95 ≤ 57 ms, 0 errors in 57,600 requests (docs/swe5001/evidence/20261016-http-baseline-dashboard/README.md); local Node workload 30 Sep, 100 closed-loop users, p95 browse 24 ms, JOIN intent 57 ms, My Orders 18 ms, 30 expiries with worst lag 58.18 s (docs/perf/sprint3-baseline.md).
- Limits to say if asked: anonymous browse only (authenticated load needs 100 real Google sessions); one monolith task, so this is not a scaling result; the ECS environment is not the SWE5006 dev host.

Hand-over *"Which brings us to scalability."*

Slide 35 Scalability: a stated trade-off, not a claim

17:03:00–17:03:30 · 30 s

Speaker Yorik · covers guideline item **5b**

Say

Scalability we state, not demonstrate. Dev is one EC2 host with its database beside it and no auto-scaling. That was deliberate: the course needs one complete system that is always on and cheap, and our proposal lists no scalability target. The same code scales out in our SWE5001 topology on ECS, but we don't claim it for this system.

≈ 59 words · ≈ 27 s at 130 wpm · slot 30 s · if short of time: If over time, drop the last sentence.

Cues

- AWS read-only 21 Oct (orchestrator): dev EC2 i-0852b6c3475b19f05 t3.large; the only EC2 Auto Scaling group is blocbuy-swe5001-nat; Application Auto Scaling targets exist only for service/blocbuy-swe5001/commerce-svc (2–6) and notification-svc (1–4), created 21 Oct 09:16 SGT. PostgreSQL 18 and Redis 8 on 127.0.0.1 of the dev host (operator guide §1).
- Why: backend ADR 0008 (one codebase, two topologies); proposal §6.3 lists maintainability, correctness under concurrency, performance, security, code quality and auditability, not scalability. Rubric 5.2: claim scalability only if auto-scaling is configured and can be triggered on stage; on dev it is neither.
- Scale path, only if asked: B10 (SWE5001 production topology) and B12 (SWE5001 V3 run: commerce 1 → 3 tasks on 19 Oct, rejected as V3 acceptance because the generator dropped iterations).

Hand-over *"And maintainability you have already seen."*

Slide 36 Maintainability: the pipeline you just saw, and the seams in the code

17:03:30–17:04:00 · 30 s

Speaker Yorik · covers guideline item **5b**

Say

Maintainability was the pipeline Pan ran twenty minutes ago: one change through six required checks, a code-owner review and a smoke test, live on this same site. In the code, our proposal's primary driver: a new pricing mode is a new strategy class plus one enum constant, a new consequence of a team's outcome is a new subscriber, and the existing rules stay untouched.

≈ 64 words · ≈ 30 s at 130 wpm · slot 30 s · if short of time: If over time, drop "and the existing rules stay untouched".

Cues

- Points back to slide 31 (rubric 5.4: do not repeat the run). If the §4 PR was not merged on air, say so and show its checks instead.
- Seams: PricingStrategy + PricingStrategyResolver (slide 20; a duplicate PricingMode fails startup); TeamSucceeded / TeamExpired subscribers (slide 18); payment providers behind the CheckoutGateway and RefundGateway ports (external-interface transition strategy, slides 10–11; we do not claim GoF Adapter for them). A new pricing mode also needs a PricingMode constant, a domain edit, so the slide says "existing rules", not "domain unedited". Proposal §6.3 "Maintainability & extensibility (primary design drivers)… new pricing schemes, payment providers, and team-resolution consequences must be addable without modifying existing domain logic". 18 ArchUnit rules keep the module boundaries on every build (ArchitectureTest.java).

Hand-over *"Thank you."*

Slide 37 Thank you 17:04 · Q&A

Speaker Yorik · covers guideline item **5b**

Say *Thank you. We're happy to take your questions.*

≈ 8 words · ≈ 4 s at 130 wpm

Cues

- 17:04:00. The buffer to 17:10 answers questions parked during the talk first (Lim reads them from the backchannel and names the owner), then new ones. Hard stop 17:10 (rubric G1).
- Scribe (rubric G9): `[CONFIRM: scribe name]` logs every lecturer question and remark verbatim; the log becomes #463 report tasks by 27 Oct.

Q&A (17:04–17:10)

Likely question	Answer: decision → reason → evidence or limitation (30–45 s)	Owner
Why is there no auto-scaling on dev?	Dev is one EC2 host running the monolith in Docker Compose, with PostgreSQL and Redis on the host (backend ADR 0008), so SWE5006 needs one complete, always-on system at low cost. Auto-scaling exists only in the SWE5001 topology on ECS (commerce 2–6 tasks on ALB requests per target, read 21 Oct), which we do not claim for SWE5006. Limitation: the dev host is a single point of failure.	Yorik → Pan
Is the refund real?	A real Stripe Sandbox refund: the Refund API in test mode, so no money moves. Locally on 20 Sep, refund re_3UHbeV… reached succeeded and a recheck created no duplicate (B11). On dev: `[FILL: Sandbox refund id from the 24 Oct dry run, or say "not yet proven on dev" (#223, #517)]` .	Yorik
What happens if Stripe is down?	The Checkout attempt is saved before Stripe is called, so a timeout leaves one recoverable attempt with a stable provider key, never a second payment (StripeCheckoutFlowTest). Refunds stay as durable obligations; the worker retries every 30 s with the same idempotency key, and after 23 hours it only searches Stripe and hands mismatches to a person.	Yorik
How do you avoid a double payment?	A repeated Checkout request with the same buyer, intent and Idempotency-Key reuses the payment. Webhook events are stored by event ID, so a replay changes nothing (StripeLocalIntegrationTest.eightConcurrentWebhookReplaysStillProduceOneMemberAndOneOrder). When paid buyers race for the last seat, one gets it and every other gets a refund obligation (StripeLocalIntegrationTest.twentyBuyersRaceForLastSeatAndNineteenRefundObligationsPersist, local).	Yorik → Eric
Why did the demo not show notifications?	By decision: the dev host runs no message broker, so the outbox writer, relay and consumer are off (application-ec2.yml; ADR 0007 scope decision, 11 Oct). Notifications run on the SWE5001 environment through SNS and SQS, and locally and in CI against ElasticMQ; slide 19 shows the relay's Template Method.	Pan
Why not race two buyers for the last seat on dev?	Because a live race is not repeatable on shared dev and would leave a refund obligation on a real account. The race is a test that runs on every pull request that changes the backend: twenty paid buyers race for the last seat of a team of two through the production intent and fulfilment ports against PostgreSQL 18 in Testcontainers; the team ends with 2 members, 2 orders, current_size 2 and 19 refund obligations (StripeLocalIntegrationTest.twentyBuyersRaceForLastSeatAndNineteenRefundObligationsPersist). It is a local test, and we say so (step D6).	Yorik → Eric
How do you know expiry happens within one minute on dev?	By design and by a local measurement, not by a dev timestamp. TeamExpirySweepJob runs every minute (fixedDelay PT1M) under a database lease and expires every open team whose deadline has passed; the local run of 30 Sep measured a worst lag of 58.18 s for 30 teams under 100-user load. On dev the demo shows the outcome: the 16:05 bloc, due ≈ 16:45, is refunded by the time we look. `[VERIFY: whether the bloc row or receipt shows the deadline and the closing time on dev]`	Yorik

Is 401 vs 403 vs 400 deliberate?	Yes. The Sa-Token interceptor checks login on every /api/v1 path except anonymous GETs of listings and invites, so no session is 401 COMMON_401_001; a signed-in buyer on an admin endpoint fails StpUtil.checkRole(ADMIN) and gets 403 COMMON_403_001; a money write without the X-BlocBuy-Request marker is 400 COMMON_400_001, and a wrong value is 403 (StripeCheckoutController.checkCsrf). Mapping: GlobalExceptionHandler.	Yorik → Liu Jin
(Questions parked during the talk)	Lim reads each parked question from the backchannel in one line and names the owner; those come first. Answer each in 30–45 s: decision, reason, evidence or limitation.	Lim

6 Timeline before, on and after the day

Before the day

Date	Do	Owner
Wed 21 Oct	Deck re-planned from the rubric; §1 drafted.	Lim
Fri 23 Oct	**Slides due** (#464). Each owner reviews their slides, cards and Q&A rows; the Sprint 4 review and retro feed slides 5 and 7. **18:00 go/no-go** for the live payment steps, posted in #464.	All; Yorik decides with Lim
Sat 24 Oct	**Full dry run on dev** of D1–D6 in order and timed: a `US34-04` bloc started ≈ 10:00 by account B expires and refunds (D4), START and JOIN on `US34-04` with both accounts (D1, D2); payment and refund ids in the day log (no secrets). **Backup videos** of both demos recorded on the final Sprint 4 build deployed to dev (Sprint 4 ended 23 Oct), 1080p, one clip per step, stored in the cloud and on a USB drive (rubric G6). **k6 pre-run** on the SWE5001 ECS environment; keep its summary for slide 34.	Yorik, Eric (system); Pan (pipeline, k6)
Sun 25 Oct	**Timed rehearsal** in Zoom at the real clock with injected questions; every diagram slide and its crops checked on a projector or TV from across the room (rubric V7); every rubric checklist box ticked; every marker resolved; rebuild deck and script.	All; Lim times

Presentation day (Mon 26 Oct)

Time (SGT)	Do	Owner
≤ 12:00	Health and recent runs (A.1); listings check; §5 preconditions list started.	Yorik, Pan
14:30	Tech check in a test Zoom: mic, camera, sharing, Do Not Disturb.	All
15:00	Merge freeze on `dev` until 17:15.	Lim
15:30	Check the §4 and §5 preconditions lists.	Pan (§4); Eric, Yorik (§5)
≈ 16:05 (≤ 16:07)	Start the expiry bloc on `US34-04` as account B and pay; bookmark the receipt. `EXPIRY OK` at 16:10, `REFUND OK` / `REFUND LATE` at 16:50.	Yorik
16:30	Start; follow the run sheet. The scribe logs every remark verbatim (rubric G9).	Lim; scribe
16:40 – 16:41	Backend pre-run PR merged; rebase the demo branch onto the new `origin/dev`, push it, then open the demo PR at 16:41.	Pan
17:04 / 17:10	Slide 37; parked questions; end.	Lim

After the presentation

- The same evening: every lecturer remark goes into #463 as a report task (by 27 Oct, rubric G9).
- Day log under `docs/agile/sprint-04/log/`: the demo PR, run links and any payment or refund ids shown (no secrets).

7 Open items before the 25 Oct rehearsal

There are 16 `[FILL]`, 6 `[VERIFY]` and 12 `[CONFIRM]` markers in this script. **FILL** is a value that does not exist yet, so never present a guess; **VERIFY** is a claim to check against the repository, the report or run logs; **CONFIRM** needs team agreement. All must be gone after the 25 Oct rehearsal.

#	Item	Owner	Blocks slide
1	Fix the Stripe webhook on dev (#517) and enable Checkout, scheduling and expiry refunds (#223); go/no-go for the live payment steps on Fri 23 Oct 18:00	Yorik	4, 33
2	Backup videos of both demos recorded by Sat 24 Oct on the final Sprint 4 build deployed to dev (Sprint 4 ends 23 Oct), stored in the cloud and on a USB drive (rubric G6, 4.6, 5.5)	Pan (pipeline), Yorik and Eric (system)	31, 33
3	Actual effort per member, agreed by all five and reconciled with the submitted progress reports (rubric 1.4); never show planned hours as actuals	Lim	5
4	Sprint 4 planned hours per member once PR #538 merges (Liu Jin 66, Eric 26, Pan 46.5)	Eric	5
5	Sprint 3 retrospective (#451): the Sprint 3 → 4 row of slide 7	Lim	7
6	Sprint 4 result, DoD step reached, retrospective and the v0.1.0-sprint4 tag date, as recorded at the 23 Oct Sprint 4 review	Eric	5, 7
7	No numbered backend ADR for: the auth change from JWT to the Sa-Token session (open decision 6); MyBatis instead of the proposal's JPA; the scheduling design; the concurrency approach (open decision 3); the team lifecycle as a transition table instead of the proposal's State; the DevSecOps changes from proposal §5.2 (OWASP Dependency-Check replaced by Trivy deps, Trivy IaC and deps not required checks, three OIDC roles split per job instead of plan/deploy/apply, two IAM users with one access key against "no static credentials"). Write them up, or cite the D-table and design docs and state the gap	Liu Jin, Eric, Pan	4, 10
8	Name a scribe for 26 Oct who logs every lecturer question and remark verbatim; turn the log into #463 tasks by 27 Oct (rubric G9)	Lim	—
9	Dev host security group allows SSH (22) from 0.0.0.0/0 although SSM sessions work (AWS read-only, 21 Oct): close it or record why it stays open	Pan	4, 29
10	Questions to Chandra: is Q&A inside the 40 min; is there a marking rubric; is the presentation graded individually (open decision 9); report due 1 Nov (briefing) or 10 Nov (#463)	Lim	—
11	Re-check slide 4 status on 25 Oct (dev health, #517, #223, gaps)	Lim	4
12	UC9 attribution: #32 and the backend design docs name Eric as implementer (PR #72); #463 gives UC9 to Liu Jin. Agree and update the matrix	Eric, Liu Jin	6
13	Report section numbers for every member's use case and design problem (rubric G8, checklist "names the report section")	Lim	6, 9–23
14	Choose the live demo change, its author and the approving code owner; open the PR before 16:42	Pan, Lim	31
15	Two Google buyer accounts (leader, joiner), their display names on screen, and the admin grant for the listings check	Yorik	33
16	SWE5001 ECS environment up for the k6 pre-run on Sat 24 Oct (it is torn down nightly); keep the k6 summary and dashboard export for slide 34; it need not be up during the talk	Pan, Eric	25, 34
17	Recount the test numbers for slide 24 on 25 Oct	Liu Jin	24

18	Reconcile frontend ADR 0004 and the EC2 operator guide with the post-merge rebuild in ci-frontend.yml and ci-backend.yml	Pan, Lim	26, 27
19	Add the media CloudFront dcsjrkmummxi7 (S3 blocbuy-media-dev-402178232802, WAF) to the deployment view: since #570 the demo listings load their covers from it	Pan	29
20	System demo preconditions for 26 Oct: Checkout and scheduling on for dev (BLOCBUY_SCHEDULING_ENABLED, #223); the 16:05 US34-04 bloc started by the Joiner account; both Google accounts signed in on Eric's machine; IDE on a local checkout of dev with StripeLocalIntegrationTest.twentyBuyersRaceForLastSeatAndNineteenRefundObligationsPersist pre-run green (Docker running); one go/no-go per step D1–D6	Yorik, Eric	32, 33
21	Use case owners for UC1, UC2, UC5 and UC7 (open decision 4): proposed UC1 Lim, UC2 Pan, UC5 Eric, UC7 Lim; agree them, or give Pan an in-scope use case instead of notifications	All	6, B15
22	Entrance choreography for the 37 re-planned slides and backups from the MOTION.md patterns (#597 choreographed the replaced #566 slides), and the SWE5006 slide numbers in MOTION.md; a follow-up PR after the content settles	Lim	all

A Appendix — check commands

Every command is read-only. Run them off the shared screen, from a checkout of `dev` at the repository root.

A.1 Health and recent runs

```
curl -s https://d1b3np88z657cp.cloudfront.net/api/actuator/health          # expect "status":"UP"
gh run list --repo PPsyrius/iss-groupbuy-team8 --workflow ci-frontend.yml --branch dev --limit 3
gh run list --repo PPsyrius/iss-groupbuy-team8 --workflow ci-backend.yml --branch dev --limit 3
gh pr list --repo PPsyrius/iss-groupbuy-team8 --base dev --state open      # what could still merge before th
e freeze
```

B Appendix — backup slides (questions only)

Backups follow slide 37 in the HTML deck and the PDF. In the deck press **B** for the index (BX) and click a row, or open `#B6` directly.

ID	Slide	Answers
B1	Agile artefacts appendix	Where are the backlog, stories, sprint plans, reviews and retros?
B2	Sprint 2 and 3 burndowns	How did earlier sprints burn down?
B3	ADR index	Which decisions are recorded, and where?
B4	Database schema, all tables	Which tables does each context own?
B5	Expiry and refund sequence	How does an expired team become refunds?
B6	UC6 refund recovery: candidates and decisions	How do you never refund twice?
B7	Test inventory	How many tests of each kind, and where do they run?
B8	Workflows, required checks and timings	What runs when, how long, and how often is it green?
B9	Known gaps and limitations	What is not done yet in the pipeline and on dev?
B10	SWE5001 production topology	What would scaling out look like?
B11	Stripe Sandbox refund, local evidence (20 Sep)	Is the refund real, and where has it been proven?
B12	Scale-out and load results (SWE5001 ECS + local baseline)	How does it scale, and how fast is it?
B13	UC9 listing statechart	Which listing moves are legal in each state?
B14	UC4 design diagrams, complete	Can we see every class and message of the join?
B15	Every member's use case and design problem	Where are the other members' models?
B16	Entity-relationship diagram, all tables	What does the whole schema look like?
D9-1	Diagram details, 1:1 (D slide-step, e.g. D14-2)	Every crop the talk steps through, readable in the PDF